



BAHIR DAR UNIVERSITY

BAHIR DAR INSTITUTE OF TECHNOLOGY

FACULTY OF COMPUTING

Industrial Project On:

Code Collab: Real-Time Collaborative Code Editor with Video Conferencing

**Submitted to the faculty of computing in partial fulfillment of the requirements
for the degree of Bachelor of Science in: Computer Science**

Group Members

Name

ID

1. Meareg TeameBDU 1506962
2. Mikyas NegaBDU 1507200
3. Markos LakachewuBDU 1506933

Advisor: MR Demeke E.

December 13, 2018 E.C

Bahir Dar University, Bahir Dar Institute of Technology

Declaration

The Project is our own and has not been presented for a degree in any other university and all the sources of material used for the project have been duly acknowledged.

<u>Name</u>	ID	Signature
1. Meareg Teame	BDU 1506962	
2. Mikyas Nega	BDU 1507200	
3. Markos Lakachewu	BDU 1506933	

Faculty: Computing

Department: Information Technology

Project Title: CodeCollab:Real-Time Collaborative Code Editor With Video Conferencing

This is to certify that I have read this project and that in my opinion it is fully adequate, in scope and quality, as a project for the degree of Bachelor of Science.

Demeke Endalie

Name of Advisor



Signature

Examining Committee Members

Signature

Date

1.Examiner 1

2.Examiner 2

It is approved that this project has been written in compliance with the formatting rules laid down by the faculty.

Acknowledgment

First and foremost, we would like to express our deepest gratitude to the **Almighty God** for providing us with the wisdom, strength, and perseverance required to complete this project

We wish to extend our sincere appreciation to our project advisor, **Demeke E.**, for their invaluable guidance, constant encouragement, and insightful feedback throughout the duration of this requirement analysis. Their expertise in software engineering principles and real-time systems significantly shaped the technical direction of this platform.

Our gratitude also goes to the **Bahir Dar Institute of Technology (BiT)** and the **Faculty of Computing** for providing the academic environment and resources necessary to facilitate our research and development.

Special thanks are due to our families and friends for their unwavering moral support and patience during the intensive hours of research and documentation. Their belief in our potential was a constant source of motivation

Finally, we would like to acknowledge the open-source community behind **FastAPI**, **React.js**, and **Monaco Editor**. The comprehensive documentation and tools provided by these communities were instrumental in the design and conceptualization of this real-time collaborative platform

List of Acronyms

API	Application Programming Interface
BiT	Bahir Dar Institute of Technology
BR	Business Rule
CPU	Central Processing Unit
E2EE	End-to-End Encryption
FREQ	Functional Requirement
HD	High Definition
HTTPS	Hypertext Transfer Protocol Secure
ICE	Interactive Connectivity Establishment
IDE	Integrated Development Environment
I/O	Input/Output
JS	JavaScript
LLM	Large Language Model
LTS	Long Term Support
MVP	Minimum Viable Product
OOAD	Object-Oriented Analysis and Design
OT	Operational Transformation
P2P	Peer-to-Peer
RAD	Requirement Analysis Document
RAM	Random Access Memory

SPA	Single-Page Application
STUN	Session Traversal Utilities for NAT
TURN	Traversal Using Relays around NAT
UC	Use Case
UI	User Interface
UML	Unified Modeling Language
VCS	Version Control System
WebRTC	Web Real-Time Communication
WSS	Web Socket Secure

List of Figures

1. Use Case Diagram	17
2. State Chart Diagram	23
3. Activity Diagram	25
4. Sequence Diagram	26

List of Tables

1. Functional Requirements (FREQ)	15
2. Business Rule Documentation	19
3. Analysis Class Model	26
4. Identifying Change Cases	31

Table of Contents

Chapter 1: Introduction

● 1.1 Background of the Project	2
● 1.2 Statement of the Problem	3
○ 1.2.1 Cognitive Load and Context Switching	3
○ 1.2.2 Latency and Synchronization Delays	3
○ 1.2.3 Fragmented and Platform-Dependent Experience	3
● 1.3 Objectives	4
○ 1.3.1 General Objective	4
○ 1.3.2 Specific Objectives	4
● 1.4 Methodology	5
○ 1.4.1 Requirement Gathering and Elicitation	5
○ 1.4.2 Analysis and Design Phase	5
○ 1.4.3 Implementation Tools and Technologies	6
○ 1.4.4 Testing and Validation	6
● 1.5 Feasibility	7
○ 1.5.1 Economic Feasibility	7
○ 1.5.2 Technical Feasibility	7
○ 1.5.3 Time Feasibility	8
● 1.6 Beneficiaries or Significance of the Project	8
○ 1.6.1 Stakeholders and Beneficiaries	8
○ 1.6.2 Significance of the Project	9
● 1.7 Limitations of the Project	9
○ 1.7.1 Scope of the Solution	9
○ 1.7.2 Challenges and Constraints	10
○ 1.7.3 Activities Left Undone	10
● 1.8 Scope of the Project	11
○ 1.8.1 System Boundaries	11
○ 1.8.2 Subsystems and Services	11
○ 1.8.3 Business Processes to be Automated	12
● 1.9 Organization of the Project	12

Chapter 2: System Features & Requirement Analysis

● 2.1 The Existing System	14
● 2.2 Proposed System	14
● 2.3 Requirement Analysis	15
○ 2.3.1 Functional Requirements (FREQ)	15
○ 2.3.2 System Use Cases	16
■ 2.3.2.1 Use Case Diagram	16
■ 2.3.2.2 Use Case Documentation	17
○ 2.3.3 Business Rule Documentation	19
○ 2.3.4 User Interface Prototype	20
■ 2.3.4.1 Low-Fidelity Prototypes (Wireframes)	20
■ 2.3.4.2 High-Fidelity Prototypes (Mockups)	21
○ 2.3.5 System States	23
○ 2.3.6 Activity Diagram	23
○ 2.3.7 Sequence Diagram	25
○ 2.3.8 Analysis Class Model	26
○ 2.3.9 Logic Model (Pseudo-code)	27
● 2.4 Non-Functional Requirements	28
● 2.5 System Requirements	29
○ 2.5.1 Hardware Requirements	29
○ 2.5.2 Software Requirements	30
● 2.6 Key abstraction with CRC analysis	30
○ 2.6.1 Conceptual Modeling	31
○ 2.6.2 Identifying Change Cases	31
○ 2.6.3 User Interface Prototyping	32

Abstract

The project, titled **Code Collab**, is a web-based real-time collaborative code editor integrated with video conferencing. Its primary purpose is to eliminate the inefficiencies of fragmented workflows by providing a unified environment for synchronous coding and communication. The system is designed to reduce cognitive load, minimize synchronization latency, and replace the "commit-and-push" bottleneck of traditional version control with real-time interaction.

The existing workflow suffers from data desynchronization and a lack of immediate feedback during remote pair programming. To address this, a solution is proposed using **FastAPI**, **React.js**, and **WebRTC** protocols. We employed various data collection and analysis methods, following an **Object-Oriented Analysis and Design (OOAD)** approach. The proposed system is analyzed in detail, and the design specifications are prepared for implementation to provide a seamless, high-performance experience for remote developers and educators

Chapter One: Introduction

1.1 Background of the Project

The global software development paradigm has undergone a significant transformation, shifting from localized, office-based teams to highly distributed, remote-first environments. In this modern landscape, the ability to collaborate in real-time is no longer a luxury but a fundamental necessity for professional software engineering, academic instruction, and technical recruitment.

Historically, collaboration was managed through asynchronous version control systems (VCS) such as Git. While VCS is indispensable for tracking long-term changes and managing code repositories, it is inherently designed for "offline" work; it does not support the immediate, synchronous interaction required for pair programming or live debugging. When developers need to work together on a single logic block simultaneously, Git's "commit-and-push" cycle becomes a bottleneck rather than a facilitator.

CodeCollab is situated at the intersection of Real-Time Distributed Systems and Full-Stack Web Development. It leverages modern web protocols to bridge the gap between static code editing and live communication. Unlike traditional IDEs that rely on local installation, CodeCollab utilizes a browser-based architecture to provide a unified workspace. By integrating high-performance text synchronization with peer-to-peer media streaming, the project moves away from the fragmented approach of using separate tools for video and code. It introduces a "living" document model where the state of the editor is consistent across all clients, allowing for a collaborative experience that mimics sitting at the same desk, regardless of physical distance.

1.2 Statement of the Problem

- The rise of remote work and distributed software teams has fundamentally changed how developers collaborate. However, the tools currently available for synchronous programming have not kept pace with these changes, leading to several critical inefficiencies:

1.2.1 Cognitive Load and Context Switching

Most existing development workflows require users to "juggle" between multiple, disconnected applications. For instance, a pair-programming session typically involves

running a heavy Integrated Development Environment (IDE) like VS Code alongside a separate video conferencing tool like Zoom or Google Meet. This constant switching between windows breaks the developer's "flow state," leads to cognitive fatigue, and reduces overall workflow efficiency. There is currently a lack of a unified, "single-pane-of-glass" solution that allows developers to communicate and code in the same environment.

1.2.2 Latency and Synchronization Delays

Traditional collaboration tools, such as basic screen sharing, offer a "driver-observer" experience rather than true collaboration. These methods suffer from significant visual lag (latency) and high bandwidth consumption, making it difficult for the observer to follow rapid code changes. Furthermore, these tools lack the immediacy of live, synchronous updates; when multiple developers attempt to discuss or modify code simultaneously, the lack of real-time state synchronization often leads to confusion and verbal "over-explaining" of simple logic changes.

1.2.3 Fragmented and Platform-Dependent Experience

The current ecosystem is highly fragmented. Many collaborative coding plugins are platform-dependent, requiring all participants to use the same IDE or version of a software suite. This creates barriers when developers with different setups need to collaborate quickly. There is a distinct absence of a high-performance, browser-based platform that merges professional-grade code editing—complete with syntax highlighting and execution—with native face-to-face communication tools that are accessible regardless of the user's local operating system or hardware configuration.

1.3 Objectives

1.3.1 General Objective

The primary goal of this project is to design and develop a robust, unified web-based platform that integrates a real-time collaborative code editor with peer-to-peer video conferencing. By merging these functionalities into a single interface, the system aims to eliminate the overhead of context switching and provide a seamless environment for remote pair programming, technical interviews, and collaborative learning.

1.3.2 Specific Objectives

- ❖ To achieve the general objective, the following specific technical goals have been identified:
 - Design a High-Concurrency Backend: Develop an asynchronous server architecture using FastAPI capable of managing multiple simultaneous collaborative sessions without data collision
 - Implement Real-Time Synchronization: Develop a synchronized text editor engine with live cursor tracking and millisecond-latency code updates using WebSockets and Socket.io.
 - Integrate Peer-to-Peer Communication: Incorporate a low-latency video and audio conferencing module using the WebRTC API, enabling direct browser-to-browser media streaming.
 - Develop a Room Management System: Build a secure session management subsystem that generates unique, shareable Room IDs to isolate collaborative environments and ensure privacy.
 - Enhance Developer Experience: Integrate the Monaco Editor to provide professional features such as multi-language syntax highlighting, auto-indentation, and themes.
 - Enable Client-Side Execution: Provide a sandboxed environment where users can execute code snippets and view output directly within the browser, facilitating immediate logic validation..

1.4 Methodology

1.4.1 Requirement Gathering and Elicitation

- ❖ To define the system's core capabilities, a combination of research-based technique will be employed
 - Literature Review: An in-depth study of existing collaborative platforms (such as VS Code Live Share and Google Docs) will be conducted to understand conflict resolution algorithms like Operational Transformation (OT)
 - Document Analysis: We will analyze technical documentations of real-time communication protocols (WebSockets and WebRTC) to determine the best data-exchange formats for code .
 - Prototyping: Iterative UI mockups will be presented to potential users to gather feedback on layout efficiency and feature placement.

1.4.2 Analysis and Design Phase

The system will be modeled using the Object-Oriented Analysis and Design (OOAD) approach. This allows for a modular architecture where components like the "Editor" and "Video Stream" are treated as independent objects

UML Modeling: Unified Modeling Language (UML) will be used to visualize system behaviors. This includes Use Case diagrams to define actor interactions, Sequence diagrams for the chronological flow of WebSocket events, and Class diagrams for the underlying data structures.

1.4.3 Implementation Tools and Technologies

- ❖ The development will utilize a modern "FastAPI + React" stack, chosen for its performance and asynchronous capabilities:
 - Frontend Development: React.js: Used to build a responsive, component-based single-page application (SPA)
 - Monaco Editor: The core text editor engine (the same power behind VS Code) will be integrated to provide professional-grade syntax highlighting and IntelliSense
 - Backend Development: FastAPI: A high-performance Python framework used to handle API requests and manage the application logic asynchronously.
 - Socket.io: Utilized to provide a reliable abstraction layer for WebSockets, ensuring real-time, bi-directional communication for code updates.
 - Real-time Communication: WebRTC API: This will be implemented to facilitate peer-to-peer (P2P) audio and video streaming directly between users, reducing server load and latency.
 - WebSockets: Used for lightweight, persistent connections to synchronize editor state and cursor positions across all clients.

1.4.4 Testing and Validation

The methodology concludes with a rigorous testing phase, focusing on Unit Testing for individual functions, Integration Testing for the connection between the frontend and the FastAPI backend, and User Acceptance Testing (UAT) to ensure the collaborative features function smoothly under varying network conditions.

1.5 Feasibility

1.5.1 Economic Feasibility

- The economic feasibility of CodeCollab is high as it primarily utilizes open-source technologies, significantly minimizing development costs
- Development Costs: By using FastAPI, React.js, and WebSockets, the project avoids expensive proprietary software licenses
- Operational Benefits: The system reduces the "Context Switching" costs identified in the problem statement, potentially saving organizations money by increasing developer productivity and reducing the need for multiple paid subscriptions for separate conferencing and editing tools.
- Infrastructure: Initial deployment can be handled through free or low-tier cloud hosting services (e.g., AWS Free Tier or Railway), making the project financially viable for a final year study

1.5.2 Technical Feasibility

- The technical requirements of the project align with the current skill set of the development team and the availability of modern web standards.
- Framework Familiarity: The team possesses foundational knowledge of Python (for FastAPI) and JavaScript (for React), which are the core languages required for implementation
- Technology Maturity: Technologies such as WebRTC for P2P video and WebSockets for real-time synchronization are mature, well-documented, and widely supported by modern browsers..
- Scalability: FastAPI's asynchronous nature ensures the backend can technically handle multiple concurrent WebSocket connections efficiently

1.5.3 Time Feasibility

- The project is structured to be completed within the academic semester timeframe by following a modular development approach
- Phase-Based Development: By prioritizing core functional requirements (FREQ-1: Real-time Editing and FREQ-3: Video Conferencing) early in the lifecycle, a Minimum Viable Product (MVP) can be delivered on schedule
- Tooling Efficiency: The use of UI prototyping tools (Figma) and rapid-development frameworks (FastAPI) allows for faster iteration and validation, ensuring that all deadlines for the Requirement Analysis Document and final prototype are met.

1.6 Beneficiaries or Significance of the Project

- ❖ The deployment of CodeCollab offers substantial benefits across various levels of the software development ecosystem. These benefits are categorized based on the stakeholders who interact with the system and the broader impact on the industry.

1.6.1 Stakeholders and Beneficiaries

- Developers and Engineers: The primary beneficiaries gain a streamlined workflow that eliminates context switching between communication and coding tools, allowing for higher focus and faster problem-solving during pair programming
- Project Managers and Tech Leads: They benefit from improved oversight and the ability to conduct real-time code reviews or "over-the-shoulder" mentoring sessions without geographical constraints.
- Recruiters and Technical Interviewers: The platform provides a professional environment for technical assessments, allowing interviewers to observe a candidate's thought process and coding style in a live, interactive setting.
- Educational Institutions: Instructors and students can use the platform for collaborative learning, enabling teachers to provide instant feedback within a student's code editor during remote lab sessions.

1.6.2 Significance of the Project

- Efficiency for Organizations: For companies, the project reduces the overhead associated with fragmented toolsets and high-bandwidth screen sharing, leading to more efficient use of technical resources and reduced latency in remote team collaboration

- **Contribution to the Project Area:** CodeCollab contributes to the field of Real-Time Systems and Collaborative Software Engineering by demonstrating an integrated approach to synchronization using FastAPI and WebSockets. It serves as a model for how modern web protocols can be used to merge high-performance text editing with peer-to-peer media streaming (WebRTC) into a single, cohesive browser-based experience.
- **Standardization of Remote Work:** By centralizing tools, the project promotes a more standardized and accessible method for remote synchronous collaboration, which is increasingly vital in the globalized software industry.

1.7 Limitations of the Project

1.7.1 Scope of the Solution

- While CodeCollab provides a robust environment for real-time interaction, it is designed as a specialized tool for synchronous collaboration rather than a full-scale production IDE. The system effectively solves the organizational problem of "Context Switching" by integrating video and code, but it is limited to the following:
- **Web-Based Execution:** Code execution is limited to standard libraries for supported languages (JavaScript, Python, C++) and does not support complex system-level operations or large-scale multi-file project compilations.
- **Session Volatility:** The current scope focuses on active collaboration sessions; thus, long-term persistent storage and version history for code are not included in the primary release.

1.7.2 Challenges and Constraints

- ★ Several factors acted as constraints during the development lifecycle:
 - **Network Latency and Stability:** High-quality WebRTC video streaming and Socket.io synchronization are heavily dependent on the users' internet bandwidth. Fluctuating network speeds in certain environments posed a challenge to achieving "zero-latency" updates.
 - **Hardware Limitations:** Real-time video encoding and decoding within a browser environment are CPU-intensive. During testing, it was observed that older hardware configurations experienced performance degradation when more than four users joined a single video conference.

- STUN/TURN Server Costs: Establishing P2P connections (WebRTC) through restrictive firewalls requires TURN servers. Due to the economic feasibility constraints of a student project, public STUN servers were used, which may not work behind all corporate firewalls.

1.7.3 Activities Left Undone

- ❖ Due to the aforementioned constraints and the fixed academic timeline, the following features were not implemented as part of this project phase:
 - Shared Terminal Access: A fully interactive, shared terminal for command-line collaboration was postponed to ensure the stability of the core code editor.
 - Advanced Project Management: Features such as task assignment boards, deadline trackers, and integrated Git repository management were excluded to focus on the real-time synchronization logic
 - End-to-End Encryption (E2EE) for Video: While signaling is secure, full E2EE for peer-to-peer media streams was not implemented in this version due to the technical complexity involved in custom encryption headers within the given timeframe

1.8 Scope of the Project

1.8.1 System Boundaries

- The scope of CodeCollab is centered on providing a high-performance, synchronous environment for small-to-medium-sized development teams or educational pairings. It focuses on web-based real-time interaction rather than replacing offline, full-featured Integrated Development Environments (IDEs)

1.8.2 Subsystems and Services

- ❖ To achieve the project objectives, the system is divided into the following functional subsystems:
 - User Management and Authentication Subsystem:
 - Services: Provides secure user registration, login, and profile management using Passport.js.
 - Services: Manages session-based authentication to ensure that only authorized users can create or join collaborative rooms.

Room and Session Management Subsystem:

- Services: Generates unique Room IDs and shareable links for session isolation.
- Services: Tracks active participants within a specific room and manages the synchronization of the room's state

Real-Time Collaborative Editor Subsystem:

- Services: Provides a high-quality code editing interface with syntax highlighting for multiple languages (JavaScript, Python, C++)
- Services: Facilitates millisecond-latency code synchronization and live cursor tracking for all participants via WebSockets (FastAPI)

Communication (Video Conferencing) Subsystem:

- Services: Enables peer-to-peer audio and video streaming directly within the browser using the WebRTC API
- Services: Provides UI controls for toggling media streams (mute/unmute, video on/off) without interrupting the coding workflow

Client-Side Execution Subsystem:

- Services: Allows users to run snippets of code and view the output within a localized console or terminal
- Services: Provides immediate feedback for testing logic and debugging during the collaborative session.

1.8.2 Business Processes to be Automated

- ❖ The project automates the following manual workflows identified in the existing system analysis:

User Management and Authentication Subsystem:

- Synchronous Pair Programming: Replaces the inefficient "screen-sharing" method with a multi-user interactive environment.
- Live Technical Interviewing: Automates the setup of a shared coding and communication environment for recruitment purposes
- Real-Time Code Review: Streamlines the process of peer-reviewing code by allowing reviewers to make live edits and comments while speaking to the author

1.9 Organization of the Project

Chapter 1: Introduction:-

- This chapter provides the foundational context of the project. It includes the background, the statement of the problem regarding fragmented developer workflows, and the objectives of the system. It also covers the feasibility study (economic, technical, and time), the significance of the project to stakeholders, and the defined boundaries and limitations of the scope

Chapter 2: Requirement Analysis

- This chapter details the core of the study. It begins with a description of the existing manual screen-sharing methods and introduces the proposed FastAPI-based solution. It identifies the functional and non-functional requirements and includes the User Interface Prototypes to show the system's intended look and feel
 - This chapter translates the requirements into technical blueprints. It utilizes UML (Unified Modeling Language) diagrams, including Use Case Diagrams for actor interactions, Activity Diagrams for process flows, and Sequence Diagrams for the chronological exchange of WebSocket messages. It also includes the Conceptual Class Diagram and the Logic Model (pseudocode) for real-time synchronization
 - This chapter summarizes the findings of the analysis phase and reflects on the potential impact of the system. It also outlines recommendations for future enhancements, such as integrating AI-driven code suggestions and persistent cloud storage for project files.

Chapter Two: System Features

2.1 The Existing System

The current workflow for remote collaborative programming in the organization relies on a combination of disparate, non-integrated tools. Typically, developers initiate a session using general-purpose video conferencing software (such as Zoom or Microsoft Teams) to share their

desktop screens while writing code in a standalone local Integrated Development Environment (IDE) like VS Code or IntelliJ.

This semi-automated approach follows a restrictive "driver-observer" model where only the user sharing the screen has direct control over the source code. The non-presenting participants are limited to viewing a compressed video stream of the editor, which often suffers from visual artifacts and significant lag due to high bandwidth consumption.

2.2 Proposed System

The proposed system is a web-based Real-Time Collaborative Code Editor designed to eliminate the synchronization delays and communication barriers inherent in traditional development environments. By utilizing a FastAPI backend and WebSocket protocol, the system enables multiple developers to edit the same source file simultaneously with millisecond-latency updates and visual cursor tracking. To ensure a comprehensive collaborative experience, the solution integrates peer-to-peer video conferencing via WebRTC and a centralized project management dashboard, directly addressing the need for seamless, remote-first technical cooperation and real-time code execution.

2.3 Requirement Analysis

2.3.1 Functional Requirements (FREQ)

ID	Requirement	Priority
FREQ-1	Users must be able to edit code simultaneously in a shared room.	High
FREQ-2	The system must broadcast cursor positions in real-time.	High

FREQ-3	Users must be able to initiate a video call within the session.	High
FREQ-4	The system must support syntax highlighting for JS, Python, and C++.	Medium

2.3.2 System Use Case

2.3.2.1 Use Case Diagram

The Use Case diagram serves as a visual representation of the system's functional scope and the interaction between external actors and the internal system modules. It defines the "who" and the "what" of the CodeCollab platform, establishing a clear boundary between the user and the automated processes.

- Actor

- Developer (Primary Actor): Any authenticated user who interacts with the system to create workspaces, write code, or communicate with team members.
- System Backend (Secondary Actor): The FastAPI server that handles the orchestration of WebSocket events and manages the room states.
- Compiler API (Supporting Actor): An external service or internal subsystem responsible for executing the code snippets and returning output

❖ Description of Use Cases

Manage Session (Create/Join Room):

- The developer can initiate a new collaborative session, generating a unique Room ID.
- Other developers can join an existing session by entering the specific Room ID, facilitating shared access to the workspace
- **Collaborative Code Editing:**

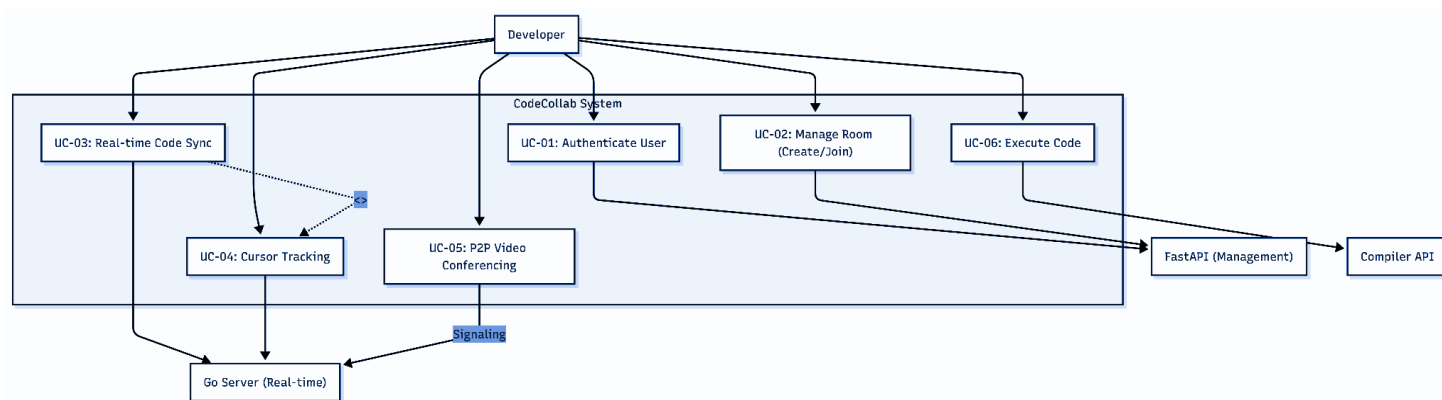
- **Synchronize Cursor:** A sub-process where the system tracks and displays the real-time position of all active users' cursors to prevent editing conflicts
- **Apply Syntax Highlighting:** An automated feature that parses the code and applies visual formatting based on the selected programming language.

Participate in Video Conference:

- Users can toggle their audio and video streams. The system manages the peer-to-peer signaling required to establish a direct communication link via WebRTC

Execute Code:

- The developer triggers the "Run" command. The system sends the current code buffer to the execution engine and displays the resulting output or error messages in the integrated console.



2.3.2.2 Use Case Documentation

❖ This section provides a deep dive into the two most critical functionalities.

• Use Case 1: Real-Time Code Synchronization

Use Case Number: UC-01

Use Case Name: Real-Time Code Synchronization

Actor: Developer

- **Description:** Allows multiple developers to edit the same source file simultaneously with real-time updates across all connected clients
- **Pre-condition:** The Developer must be successfully connected to a valid Room ID via WebSocket.
- **Post-condition:** The code buffer is identical across all participant screens; cursor positions are updated.

Priority: High

❖ **Basic Course of Action:**

- The Developer types or deletes a character in the Monaco Editor
- The Client-side application captures the "change event" and cursor coordinates.
- ❖ The Client sends a code_change payload to the FastAPI backend via WebSockets
- ❖ The Server receives the payload and identifies the specific Room ID.
- ❖ The Server broadcasts the update_editor message to all other active clients in that room. The receiving Clients' editors update their text buffers and move the colored cursor of the sender.
- ❖ Alternative Flow: *Connection Loss*: If the WebSocket disconnects, the system notifies the user and attempts a heartbeat reconnection while disabling the editor to prevent desynchronization.

- **Use Case 2: Peer-to-Peer Video Conferencing**

Use Case Number: UC-02

Use Case Name: Participate in Video Conference

Actor: Developer

- ❖ **Description:** Establishes a direct video and audio link between users in the workspace to facilitate verbal communication while coding.
- ❖ **Pre-condition:** Camera and Microphone permissions must be granted by the browser.
- ❖ **Post-condition:** Local and remote media streams are rendered in the UI grid.
- ❖ **Priority:** High
- ❖ **Basic Course of Action:**
 - The Developer clicks the "Join Call" button in the sidebar
 - The System requests access to the user's media devices (Camera/Mic)

- The System initiates a WebRTC "Offer" through the FastAPI signaling server.
- The remote peers receive the offer and send back an "Answer" via the server..
- A direct Peer-to-Peer (P2P) connection is established.
- The UI dynamically updates the grid to display the video feeds of all participants
- **Alternative Flow:**
- *Permission Denied:* If the user denies camera access, the system falls back to "Audio Only" mode or displays a warning message.

2.3.3 Business Rule Documentation

As per the template, these are the rules governing the operation

ID	Rule Name	Description
BR-01	Session Isolation	Code changes and video streams must only be broadcast to users with the matching Room ID.
BR-02	Room Capacity	A single room should support a maximum of 6 concurrent video streams to maintain performance.
BR-03	Conflict Resolution	The system uses Operational Transformation (OT) or a "last-in" rule to resolve simultaneous edits on the same line.

2.3.4 User Interface Prototype

The user interface for CodeCollab is designed with a "Single-Pane-of-Glass" philosophy. This approach minimizes the cognitive load identified in the Problem Statement (Section 1.2) by integrating complex real-time communication tools directly into the coding environment. The prototypes presented below serve as a visual contract between the user requirements and the technical implementation.

2.3.4.1 Low-Fidelity Prototypes (Wireframes)

- The low-fidelity prototypes provide a structural blueprint of the CodeCollab platform. By removing visual design elements like colors and fonts, these wireframes allow stakeholders to focus strictly on the navigation flow, information hierarchy, and functional placement.

→ UI-1: Project Dashboard (Entry Point)

- The Dashboard serves as the primary gateway for the developer. Its structural layout is designed for rapid navigation and includes:
 - Header Bar: Contains the user profile, notification icon, and global search.
 - Main Action Area: A prominent "Create New Room" button that triggers the Room ID generation logic
 - Active Rooms List: A tabular or card-based layout showing rooms the user is currently a member of, including metadata such as "Number of Participants" and "Last Modified Date."
 - Join Session Input: A dedicated text field for users to paste a unique Room ID to immediately enter a peer's workspace.

→ UI-2: Collaborative Editor Layout (The Workspace)

The workspace wireframe illustrates the spatial organization of the "Single-Pane-of-Glass" design. It is divided into three major structural columns:

- Left Sidebar (File Explorer): A collapsible tree-view navigation pane for managing files and directories within the current room session.
- Central Canvas (Code Editor): The largest structural block, dedicated to the Monaco Editor. It includes a tab bar at the top for multi-file editing and a gutter for line numbers.

- Right Sidebar (Collaboration Suite): A multi-functional panel that reserves space for:
- Participant List: A vertical list of avatars showing who is online.
- Video Feed Placeholders: Rectangular wireframes where the WebRTC video streams will be rendered
- Chat/Console Toggle: A bottom section within the sidebar to switch between a text-based chat and the code execution output.

2.3.4.2 High-Fidelity Prototypes (Mockups)

- The high-fidelity mockups represent the finalized visual design of CodeCollab. These designs incorporate a specific color palette (Deep Charcoal and Electric Blue), intuitive iconography, and dynamic UI elements that respond to real-time events.

UI-3: Collaborative Coding Workspace (The "Live" Editor)

- This mockup illustrates the system in an active state with multiple participants. The design focus is on "Presence Awareness"—ensuring every user knows exactly what their peers are doing without verbal communication.
- Themed Monaco Editor: The editor is shown using a customized "VS Code Dark" theme for high-contrast syntax highlighting.
- Dynamic Cursors: Each participant is assigned a distinct, high-visibility color (e.g., User A is Neon Green, User B is Sunset Orange). The mockup shows floating name tags attached to these cursors that fade slightly after inactivity to prevent visual clutter.
- Activity Heatmap: The line number gutter includes small color-coded indicators showing which lines have been recently modified by which user.
- **Real-time Status Bar:** At the bottom of the editor, a status bar displays the current language (e.g., Python), the number of active collaborators, and the "Synced" status icon

UI-4: Integrated Video Conference Overlay

- ❖ This mockup demonstrates how the communication layer sits atop the technical workspace. The UI is designed to be non-intrusive, allowing for "Face-to-Face" interaction without sacrificing screen real estate for code.
 - Floating Thumbnail Grid: Instead of a rigid sidebar, video feeds appear as rounded, high-definition thumbnails. These can be "pinned" to the sidebar or allowed to float as an overlay depending on user preference.

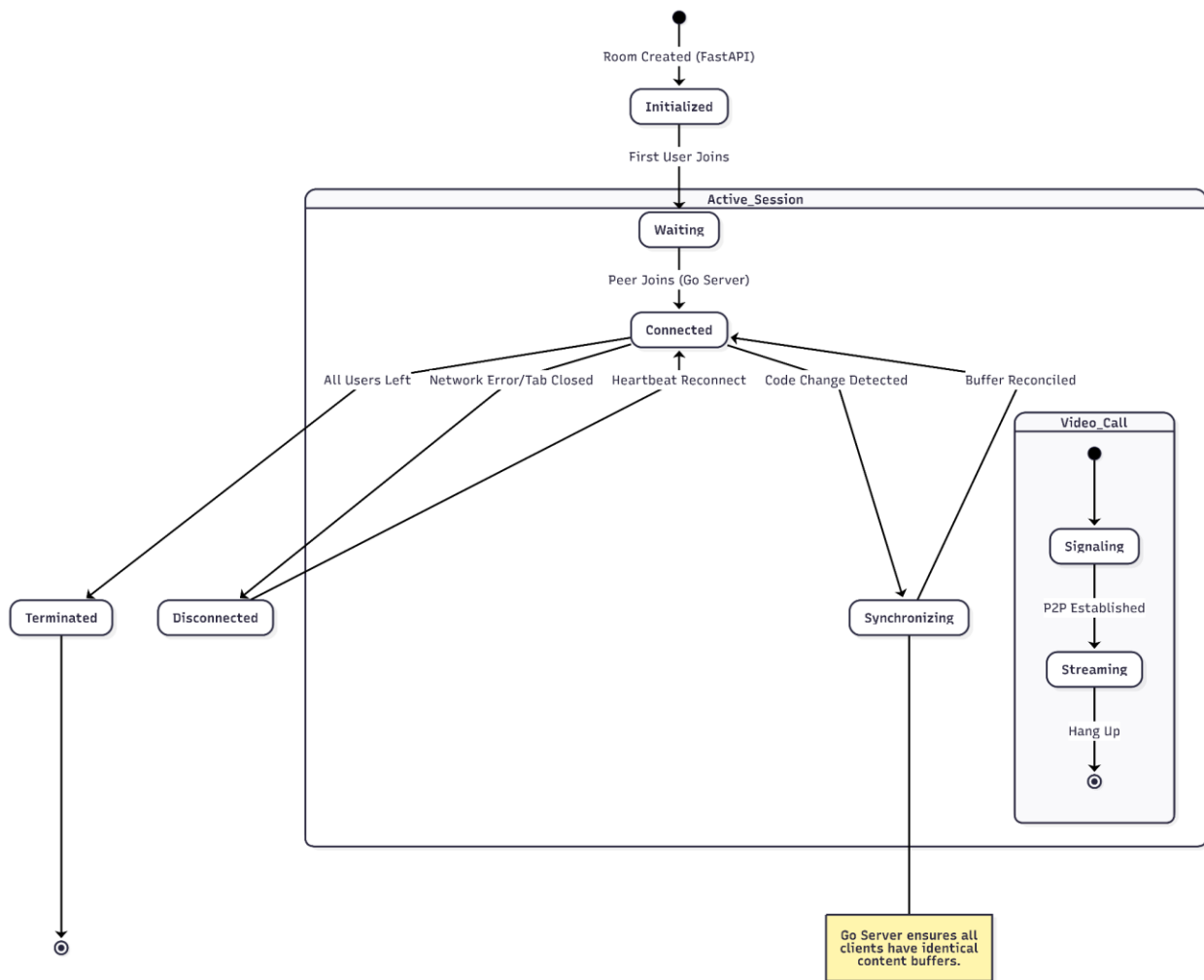
- Active Speaker Highlighting: The thumbnail of the person currently speaking is outlined in Electric Blue, helping users identify the speaker in larger groups.
- Media Control Bar: A sleek, semi-transparent toolbar at the bottom of the video grid provides quick-access icons for:
- Microphone Toggle: Mute/Unmute with a red "slashed" icon for muted states.
- Camera Toggle: Enable/Disable video feed.
- Screen Share: A dedicated button to switch from the shared editor to a full desktop stream if necessary.
- P2P Connection Quality Indicator: A small signal-strength icon on each video tile, derived from WebRTC statistics, to inform users of potential lag or connectivity issues.

2.3.5 State Chart Diagram: Room Session

- A state chart diagram describes the lifecycle of a specific object—in this case, the Room Session. It shows how the session transitions from being created to being active and, finally, closed

States:

- Initialized: The room is created on the server, and a unique ID is generated
- Waiting: The first user has joined and is waiting for others
- Active: Multiple users are connected, and real-time streams (code and video) are flowing
- Synchronizing: Triggered when a new user joins or a network lag occurs; the system pauses to ensure all clients have the same code buffer
- Terminated: All users have left, and the server deallocates the room's memory

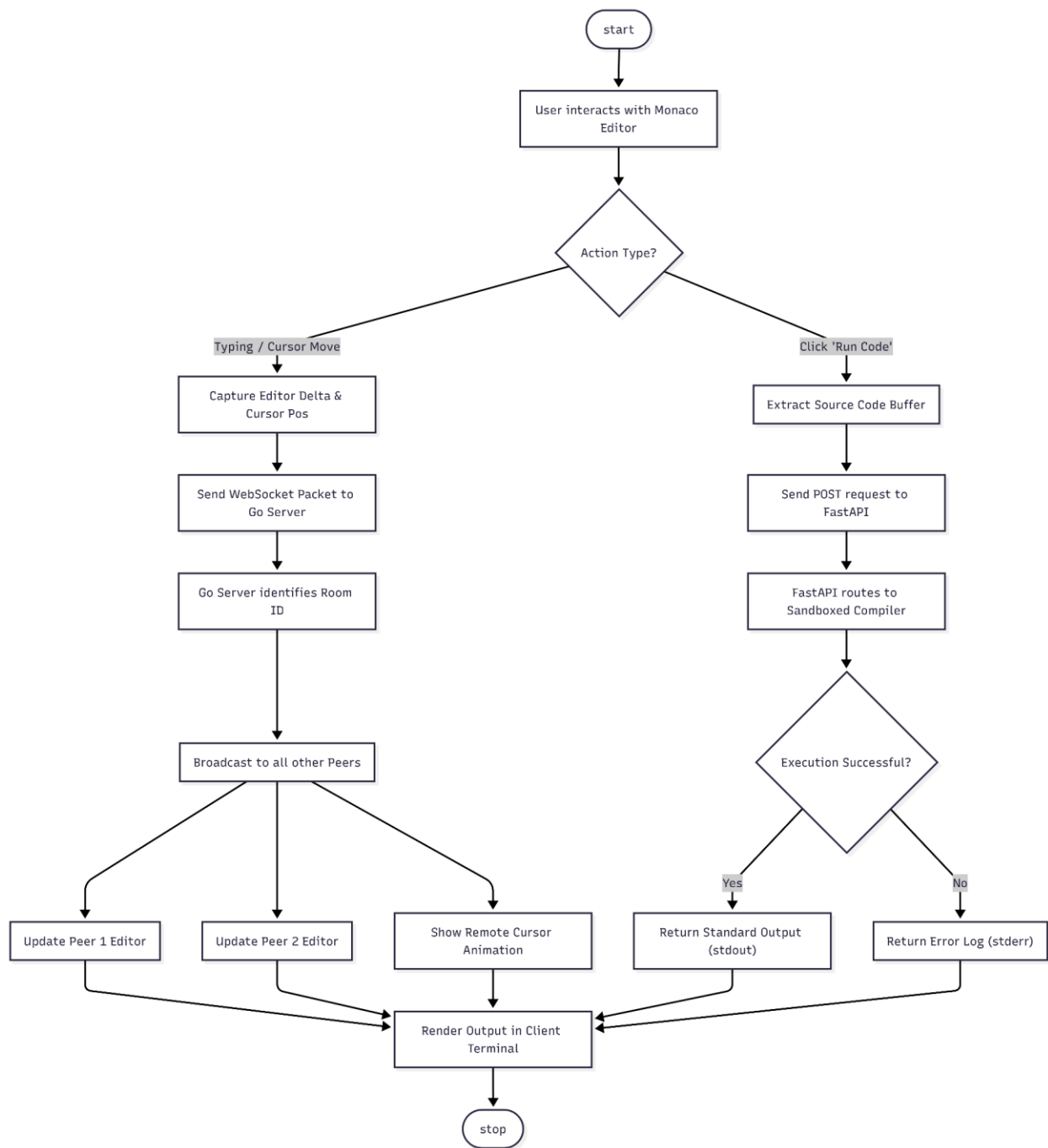


2.3.6 Activity Diagram

- The activity diagram represents the workflow of a specific use case or business process. Below is the flow for the Collaborative Editing Process

Process Flow:

- User Action: The developer types code into the editor
- Local Update: The Monaco Editor updates the local UI immediately for zero-latency feel
- Emission: The system packages the change (delta) and emits it via Socket.io
- Server Logic: The Node.js server identifies the Room ID and broadcasts the delta to all other connected peers
- Synchronization: Remote clients receive the delta and apply the change to their specific line and column



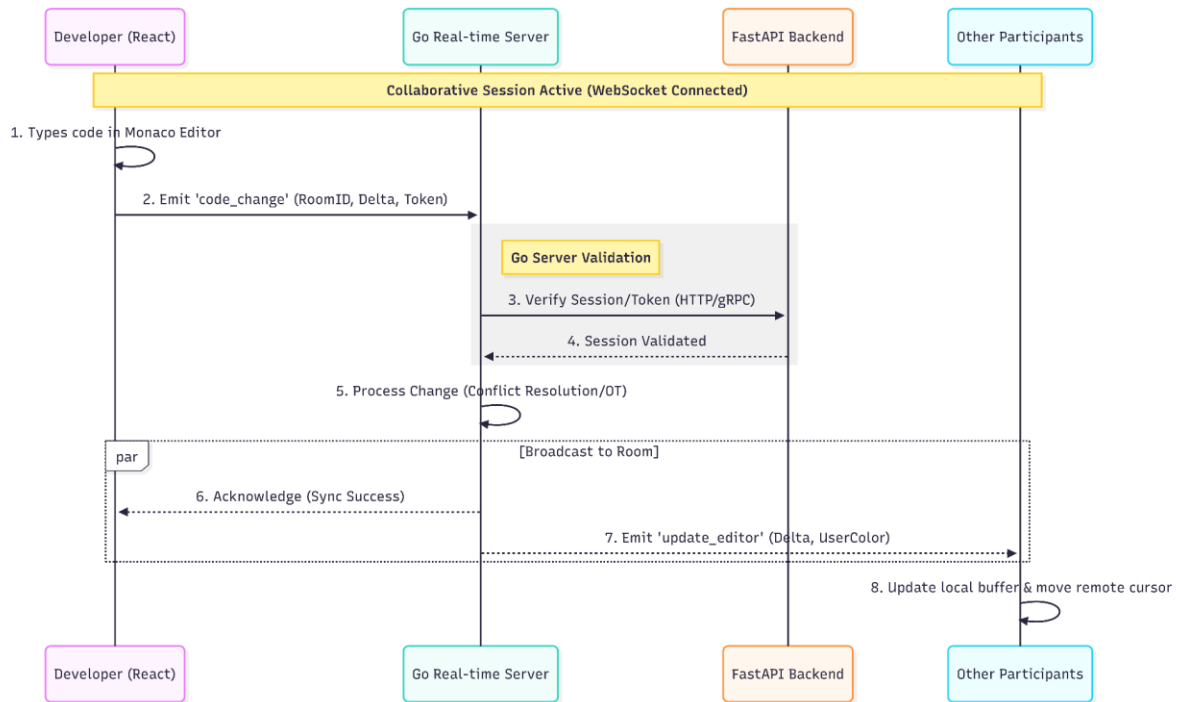
2.3.7 Sequence Diagram

- ❖ The sequence diagram shows how objects operate with one another in chronological order to respond to the actors. This is the sequence for Establishing a WebRTC Video Call

Sequence of Interactions:

- Initiator: Developer A clicks "Start Video" in the React UI
- Signaling: The system creates an "Offer" and sends it to the FastAPI Signaling Server

- Relay: The Server relays the "Offer" to Developer B
- Response: Developer B creates an "Answer" and sends it back through the server to Developer A
- ICE Candidates: Both peers exchange network information (ICE candidates) until a direct P2P connection is established
- Stream: Media streams flow directly between peers, bypassing the server to reduce latency



2.3.8 Analysis Class Model

- This model identifies the core objects, their attributes, and their relationships within the system

Class	Attributes	Methods
Room	roomID, participantList, activeLanguage	createRoom(), joinRoom(), removeUser()

Editor	contentBuffer, cursorPos, theme	updateContent(), syncCursor(), highlightSyntax()
User	userID, userName, streamID	toggleAudio(), toggleVideo(), sendMessage()
Compiler	sourceCode, inputData	executeLocally(), formatOutput()

2.3.9 Logic Model (Pseudo-code)

❖ The logic for the real-time synchronization on the server side

```

ON 'code_change' FROM client:
  GET roomId FROM socket.data
  GET changeData FROM eventPayload
  VALIDATE user membership in roomId
  IF valid:
    BROADCAST 'update_editor' WITH changeData TO (roomId) EXCEPT sender
  ELSE:

```


SEND 'error' TO sender

2.4 Non-Functional Requirements

Reliability

- The system must ensure high availability and data integrity during active collaboration sessions.
- State Consistency: The system must implement a "Conflict Resolution" mechanism to maintain synchronization even during minor network fluctuations or packet loss. If a user's connection is momentarily interrupted, the client-side state must automatically reconcile with the server's "Source of Truth" upon reconnection
- Fault Tolerance: The backend (FastAPI) should handle unexpected WebSocket disconnections gracefully, ensuring that a single user's crash does not affect the stability of the entire room.

Security

- Data privacy and secure communication are paramount, especially when proprietary code is being discussed.
- Secure Signaling: While video/audio is transmitted Peer-to-Peer (P2P), the signaling process (exchanging connection metadata) must be handled over HTTPS and WSS (Secure WebSockets) to prevent "man-in-the-middle" attacks
- Room Privacy: Access to collaborative spaces must be restricted to users possessing the unique, cryptographically strong Room ID.
- Media Privacy: The system must strictly adhere to browser-level security policies, requiring explicit user consent before activating the camera or microphone for every session.

Performance

- To mimic the experience of working on the same machine, the system must prioritize low-latency execution.
- Synchronization Latency: Text updates and cursor movements should reflect across all connected clients within a threshold of 100ms to 200ms under standard broadband conditions.
- Concurrent Scaling: The FastAPI backend must utilize asynchronous I/O to handle at least 50 concurrent rooms without a noticeable degradation in message delivery speed.

- **Resource Efficiency:** The frontend must be optimized to ensure that the integrated video conference does not consume excessive CPU cycles, allowing the Monaco Editor to remain responsive and lag-free during typing..

2.5 System Requirements

2.5.1 Hardware Requirements

- Since the platform handles real-time video encoding and high-frequency WebSocket data, the hardware must be capable of processing multi-threaded browser tasks without significant thermal throttling.

Client-Side (User) Requirements:

- **Processor:** Dual-core 2.0 GHz or higher (Intel i3/Ryzen 3 equivalent).
- **Memory (RAM):** 4GB Minimum (8GB Recommended) to handle multiple Chrome tabs and high-definition video streams concurrently.
- **Peripherals:** 720p HD Webcam and a functional microphone for the WebRTC video conferencing module.
- **Network:** Stable internet connection with at least 2 Mbps upload/download speed for smooth P2P video performance.

Server-Side (Host) Requirements:

- **Processor:** 1 vCPU (2 vCPUs recommended for handling asynchronous FastAPI workers)
- **Memory (RAM):** 1GB to 2GB for the backend service and WebSocket message broker.
- **Storage:** 500MB of available disk space for the application code and environment dependencies.

2.5.2 Software requirements

- ❖ The software requirements focus on the environment needed to interpret modern JavaScript and handle persistent network connections.

→ Client-Side (User) Requirements:

- **Web Browser:** Modern versions of Google Chrome (v80+), Mozilla Firefox (v75+), or Microsoft Edge. These browsers provide the native WebRTC API and WebSocket support required for the system.

- Operating System: Platform-independent (Windows 10/11, macOS, or Linux) as the application runs entirely within the browser.
- Development & Server-Side Software:
- Operating System: Linux (Ubuntu 20.04 LTS recommended) or Windows with WSL2.
- Runtime Environment: Python 3.9+ for the FastAPI backend and Node.js 16+ for the React.js frontend development.
- Database/Storage: Redis (optional, for WebSocket session scaling) or a lightweight SQL database (PostgreSQL/SQLite) for user management
- Development Tools: VS Code (for development), Git (for version control), and Postman (for API testing)

2.6 Key abstraction with CRC analysis

2.6.1 Conceptual Modeling: Class Diagram

Key Class Definitions:

- ❖ Room: The central container. It has a roomID and manages a collection of User and Document objects
- ❖ User: Represents a participant. Attributes include SocketID, cursorPosition, and isMicMuted. It interacts with the VideoStream class.
- ❖ Document: Contains the actual code buffer. It has methods like applydata() and getcontent()
- ❖ SignalingServer: A specialized class (singleton) that manages the WebRTC handshake between two User instances

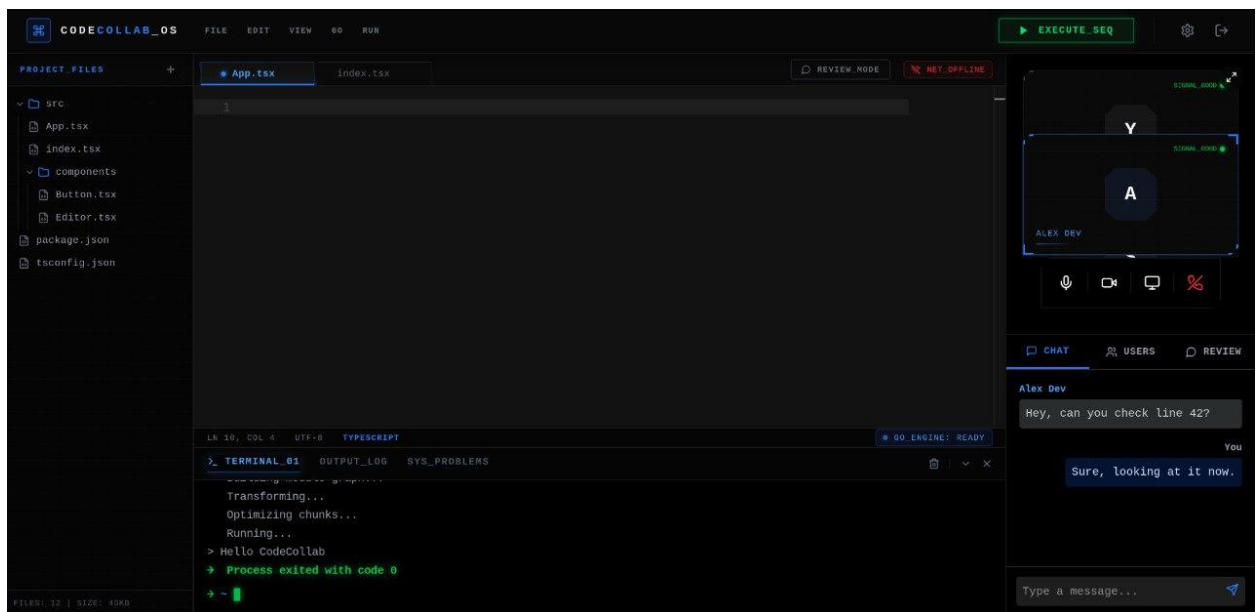
2.6.2 Identifying Change Cases

Potential Change	Likelihood	Impact

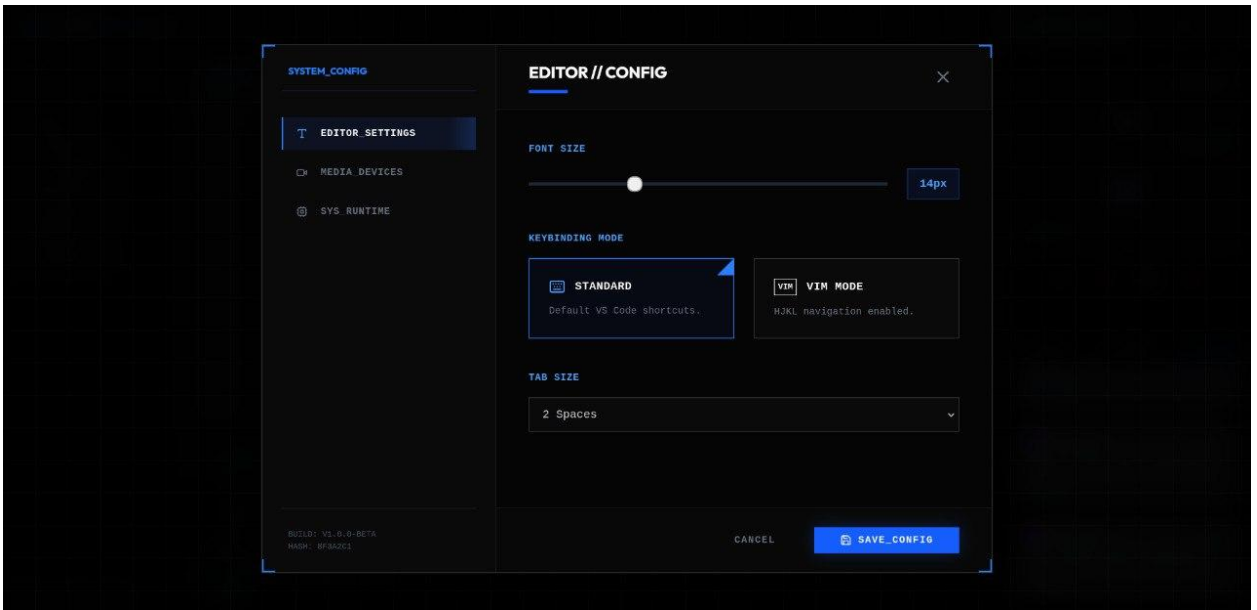
Persistent Storage	High	Medium; requires adding a Database layer (MongoDB) to save code history.
Whiteboard Integration	Medium	High; requires a new canvas-based synchronization module.
AI Pair Programmer	Medium	High; requires integration with an LLM API (like Gemini or OpenAI) to provide code suggestions.

2.6.3 User Interface Prototyping

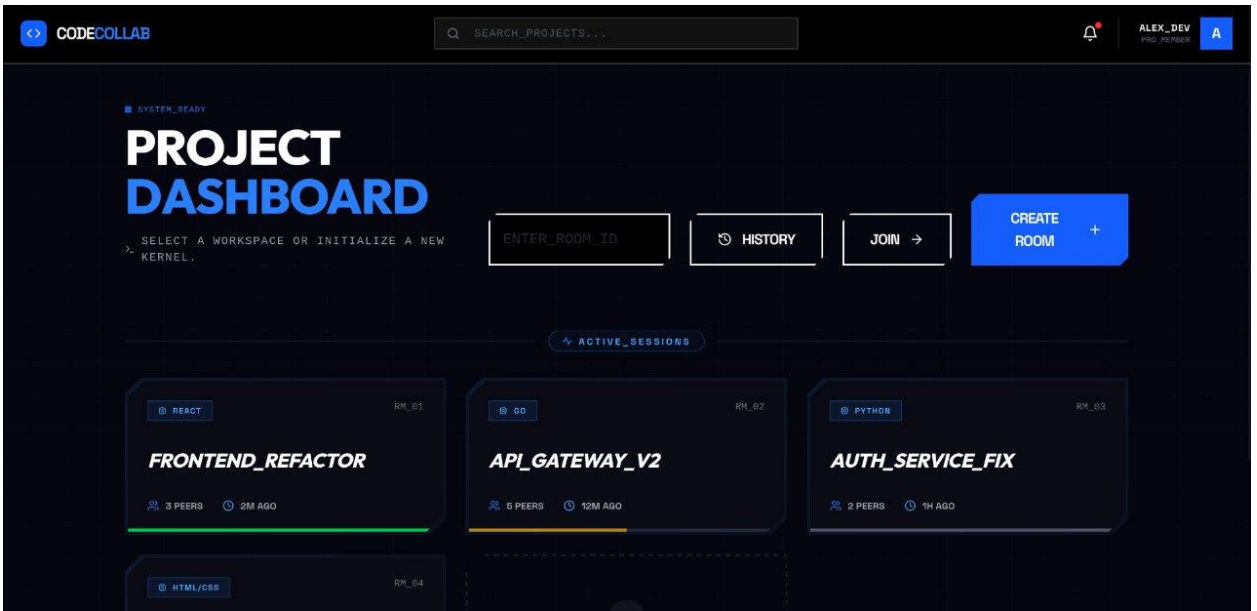
Editor UI



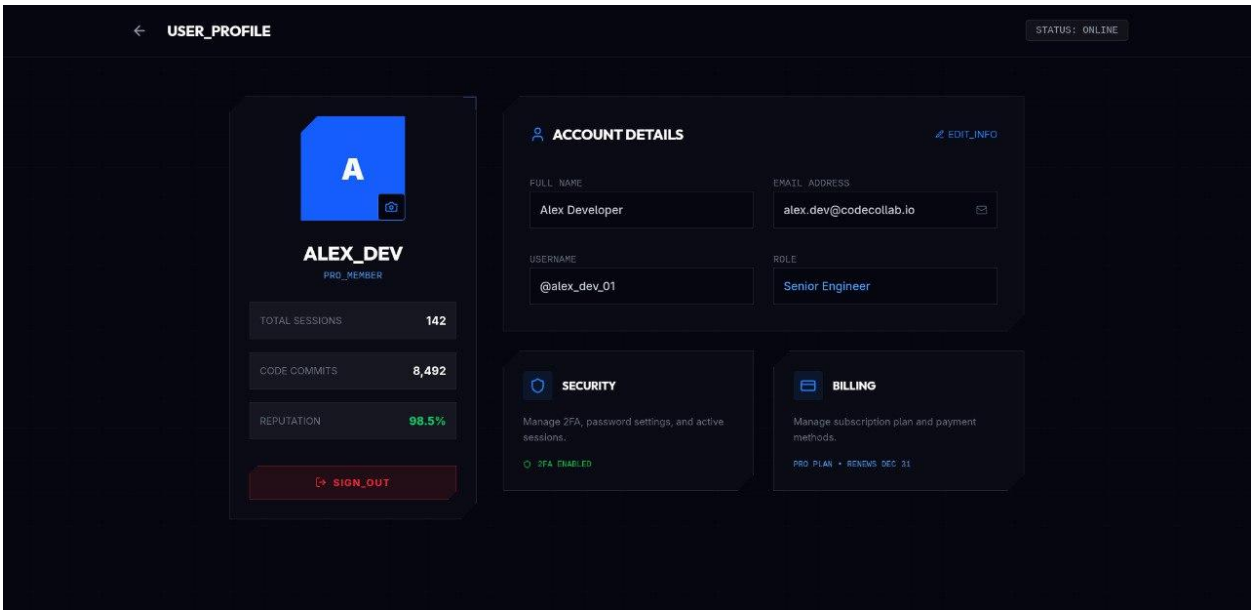
Setting UI



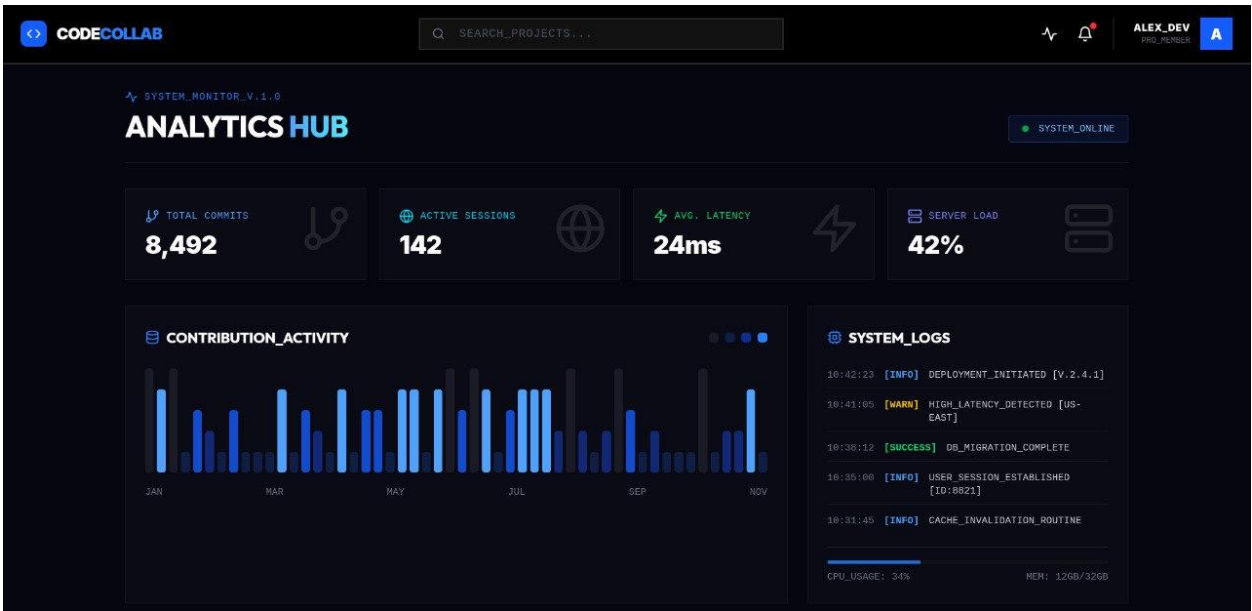
Dashboard UI



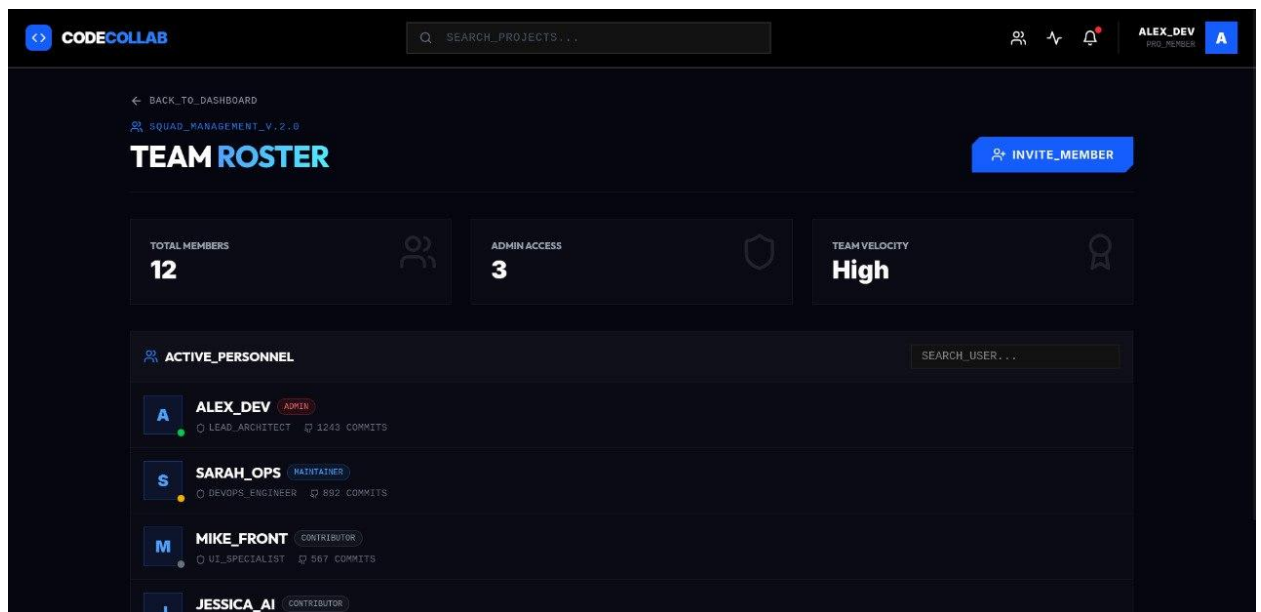
User Account UI



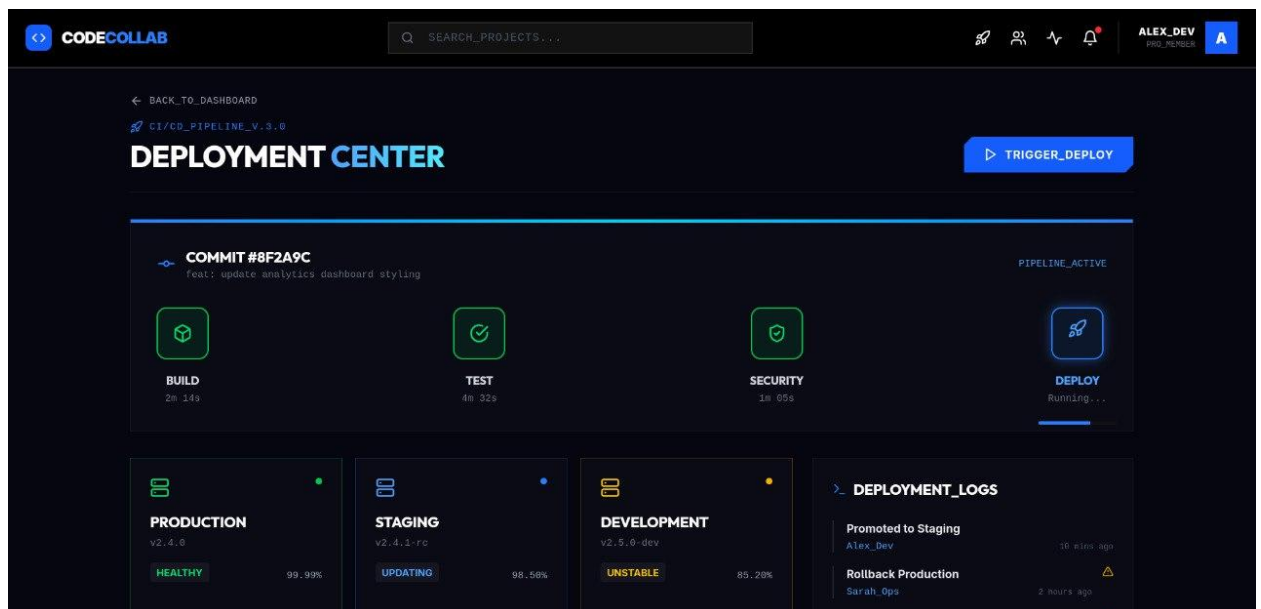
Analytics Hub UI



Team Management UI



Deployment UI



References

Socket.io (2024). Real-time bidirectional event-based communication. Retrieved from <https://socket.io/>

WebRTC.org (2024). Real-time communication for the web. Retrieved from <https://webrtc.org/>

Chapter 3: System Design

3.0 Introduction

This chapter presents the detailed **system design** of the proposed **CodeCollab** platform. System design is a critical phase in software engineering, as it transforms system requirements into a technical blueprint that guides implementation. The design defines how system components are structured, how data flows between them, and how non-functional requirements such as performance, security, and scalability are addressed.

Code Collab is a **web-based real-time collaborative coding system** designed to support distributed software development, online programming education, and technical interviews. Traditional collaboration approaches rely heavily on asynchronous tools such as version control systems, which are not suitable for synchronous collaboration scenarios like pair programming or live debugging. CodeCollab addresses this limitation by introducing a unified browser-based environment that combines real-time code editing with integrated audio and video communication.

This chapter describes the architectural structure of CodeCollab, its component organization, deployment strategy, detailed object-oriented design, persistent data modeling, user interface structure, and access control mechanisms. The goal is to demonstrate that the proposed design is robust, scalable, secure, and aligned with modern software engineering best practices.

Design Goals and Criteria

The design of CodeCollab is guided by a set of non-functional requirements that influence architectural and technological choices. These criteria ensure that the system meets both technical and user-oriented expectations.

Performance

Performance is a primary design concern for CodeCollab due to its real-time nature. Code synchronization must occur with minimal latency to ensure a seamless collaborative experience. To achieve this, the system employs **WebSocket-based communication**, which enables persistent, bidirectional connections between clients and the server. This approach avoids the overhead of repeated HTTP requests and allows immediate propagation of code changes.

Audio and video communication are implemented using **WebRTC**, which establishes direct peer-to-peer connections between users. By avoiding server-side media relaying, WebRTC significantly reduces latency and bandwidth consumption. The backend is developed using **Fast API**, an asynchronous Python framework optimized for high concurrency, allowing the system to efficiently manage multiple simultaneous collaboration sessions.

Dependability

Dependability refers to the system's ability to operate reliably under varying conditions. CodeCollab ensures dependable operation by maintaining consistent session states across all participants. The backend session manager coordinates user actions and ensures that code updates are applied in the correct order.

Persistent storage is used to retain user information, project data, and session metadata. This allows the system to recover gracefully from temporary failures such as network disruptions. Additionally, error-handling mechanisms and client reconnection strategies ensure that users can rejoin collaboration sessions without losing progress.

Scalability

Scalability is essential to support growing numbers of users and concurrent collaboration sessions. The system is designed with **stateless backend services**, allowing multiple instances to run in parallel. Containerization using Docker enables easy replication and deployment of services across cloud infrastructure.

The use of WebRTC for media streaming offloads a significant portion of network traffic from the server, further enhancing scalability. Cloud-based deployment allows the system to dynamically allocate resources based on demand.

Usability (End-User Focus)

Usability is a core design objective of CodeCollab. The system is entirely browser-based, eliminating the need for users to install or configure local development environments. This design choice lowers the barrier to entry and enables quick onboarding.

The integration of code editing, audio, and video communication within a single interface minimizes context switching and enhances productivity. The use of the Monaco Editor provides a familiar development experience, reducing cognitive load for users.

3.1 Architectural Design

The architecture of CodeCollab follows a **distributed, service-oriented architectural style**, in which functionality is decomposed into independent yet cooperating subsystems. This approach promotes modularity, maintainability, and scalability.

The system architecture is composed of the following subsystems:

- Frontend Subsystem
- Backend Subsystem
- Real-Time Communication Subsystem

- Database Subsystem

Each subsystem encapsulates a well-defined set of responsibilities and communicates with other subsystems through standardized interfaces. The architectural design emphasizes **high cohesion**, ensuring that closely related functionalities are grouped together, and **low coupling**, allowing subsystems to evolve independently.

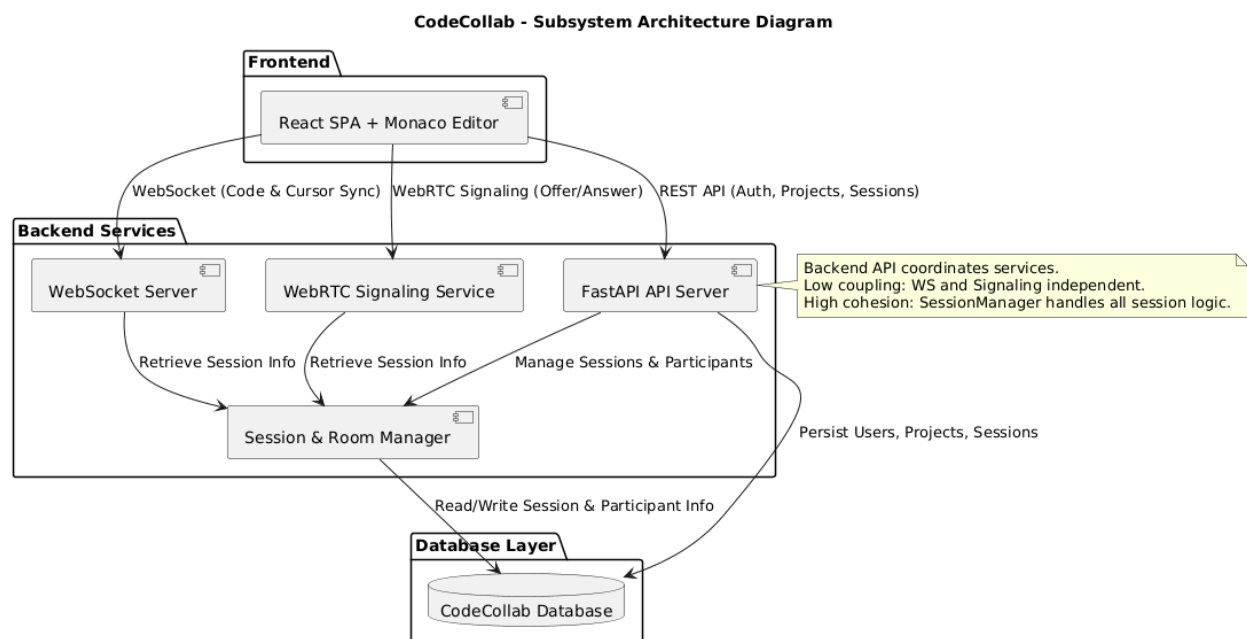
Subsystem Interaction and Responsibilities

The **Frontend Subsystem** is responsible for presenting the user interface and capturing user interactions. It initiates communication with backend services and manages real-time collaboration features.

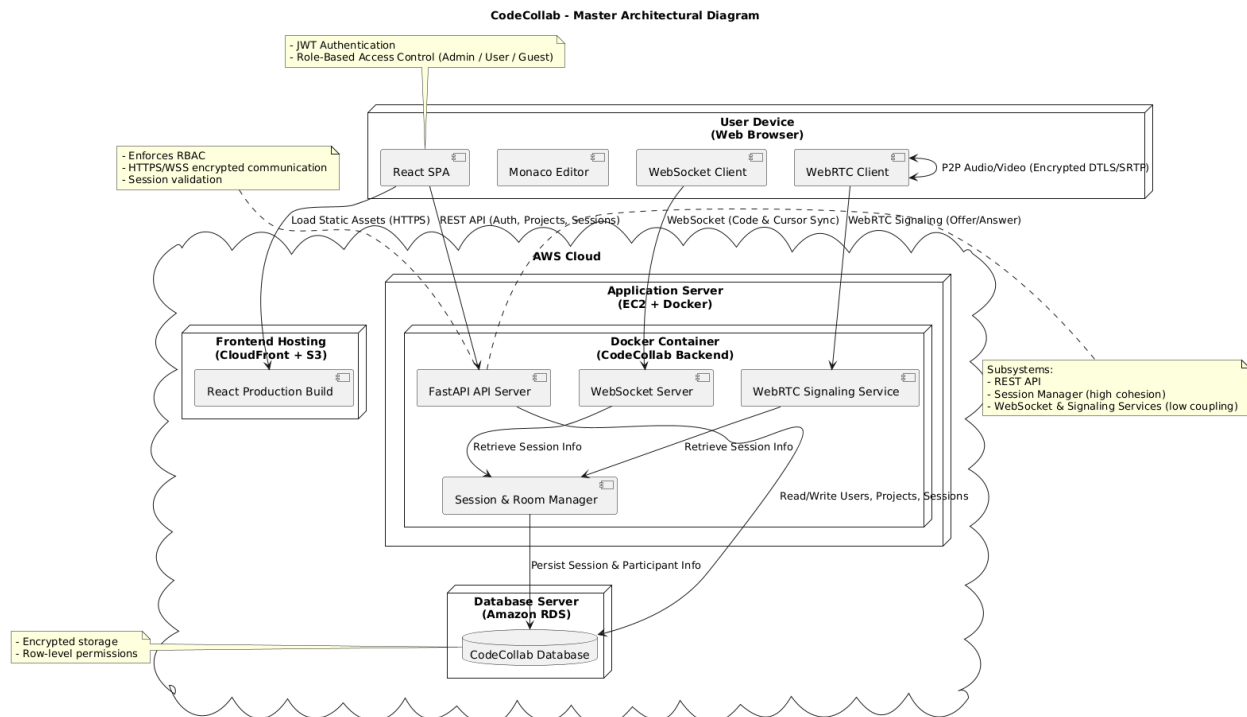
The **Backend Subsystem** acts as the central coordinator of the system. It handles authentication, authorization, project management, session orchestration, and access control. It also mediates interactions between the frontend and the database.

The **Real-Time Communication Subsystem** supports synchronous collaboration through WebSocket-based code synchronization and WebRTC signaling for audio and video communication.

The **Database Subsystem** provides persistent storage and ensures data integrity for user accounts, projects, sessions, and collaboration artifacts.



Architecture Diagram



3.1.1 Component Modeling

Component modeling decomposes each subsystem into smaller, reusable software components. This design approach supports separation of concerns and improves system maintainability.

Frontend Components

The frontend is implemented as a **React Single Page Application (SPA)** and consists of several components:

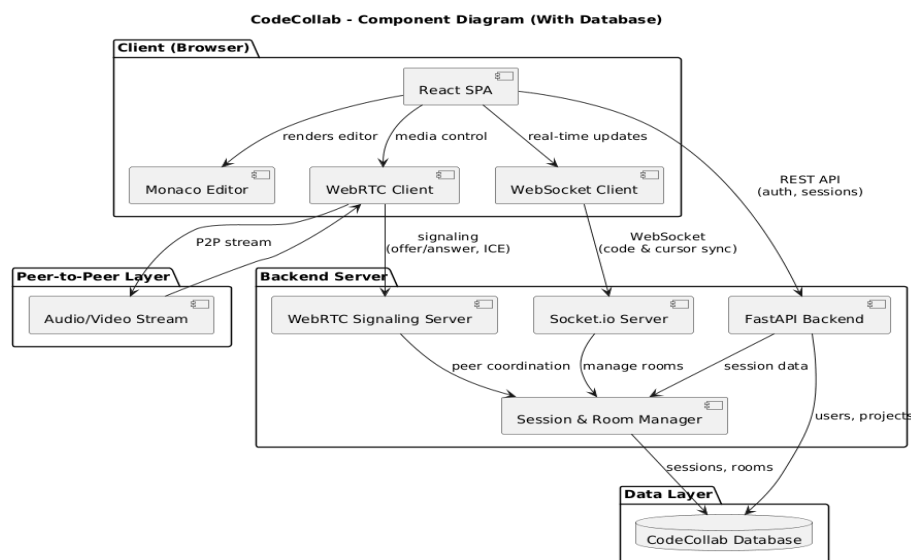
- ✓ **React SPA Core:** Manages application routing, state management, and UI rendering.
- ✓ **Monaco Editor Component:** Provides advanced code editing features such as syntax highlighting, auto-completion, and error detection.
- ✓ **WebSocket Client Component:** Handles the transmission and reception of real-time code updates.
- ✓ **WebRTC Client Component:** Manages audio and video streams and handles peer connection establishment.

Backend Components

The backend is composed of multiple components that collaborate to deliver system functionality:

- ✓ **FastAPI API Server:** Exposes RESTful endpoints for authentication, user management, and project operations.
- ✓ **Session and Room Manager:** Maintains session lifecycle, participant lists, and collaboration states.
- ✓ **WebSocket Server:** Manages real-time code synchronization events.
- ✓ **WebRTC Signaling Service:** Coordinates signaling messages required for peer-to-peer communication.

Each component interacts through clearly defined interfaces, ensuring modularity and flexibility.



3.1.2 Deployment Modeling

The deployment model describes how software components are mapped onto physical and virtual infrastructure.

Client-Side Deployment

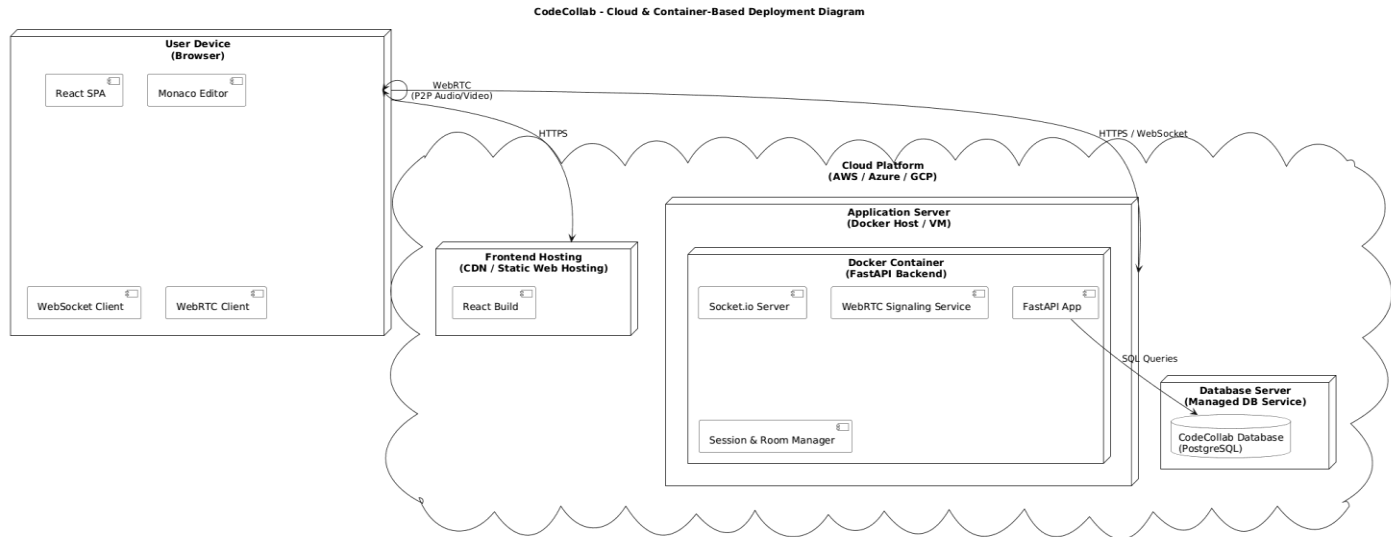
Client-side components execute within a standard web browser. This includes the React application, Monaco Editor, WebSocket client, and WebRTC client. Secure communication protocols ensure safe interaction with backend services.

Server-Side Deployment

Backend services are deployed on cloud-based virtual machines and run inside Docker containers. Containerization ensures environment consistency, simplifies deployment, and supports rapid scaling. Backend components communicate internally using low-latency connections.

Database Deployment

The database subsystem is deployed using a managed cloud database service. Access is restricted to backend services through secure network configurations. Regular backups and redundancy mechanisms ensure data availability and durability.



3.2 Detail Design

Detail design focuses on the internal structure and behavior of the system. It defines object-oriented classes, their interactions, and how data is persisted.

3.2.1 Design Class Model

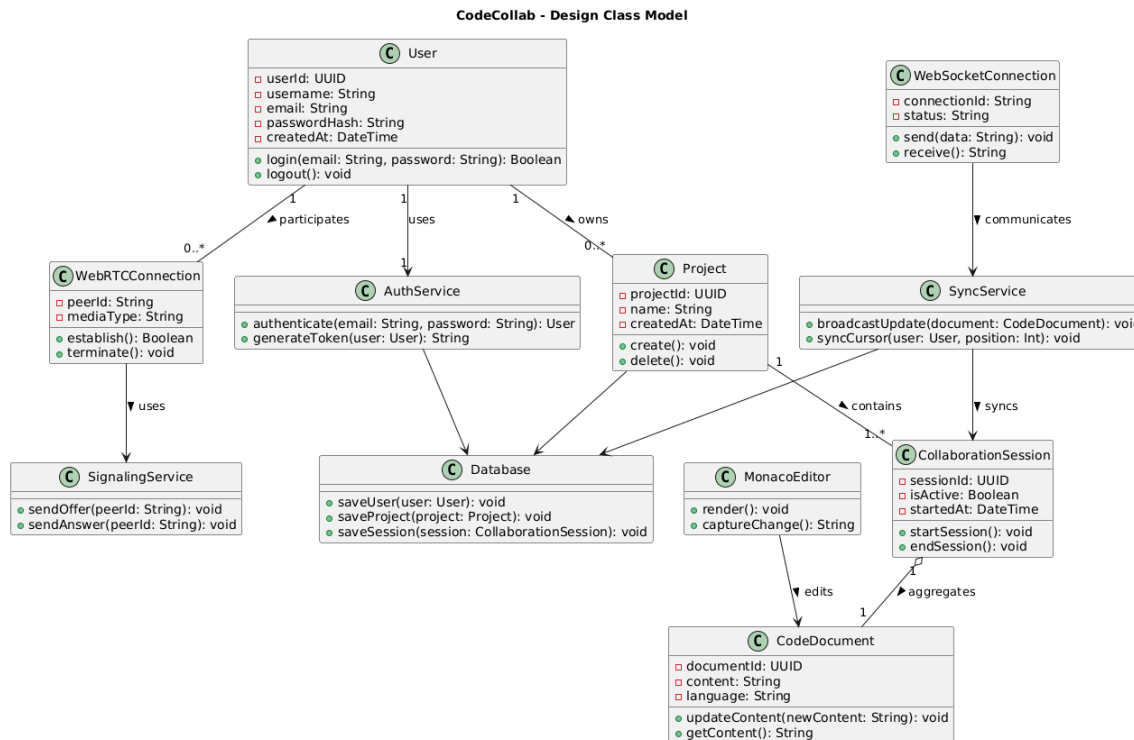
The design class model represents the system's object-oriented structure and defines how responsibilities are distributed among classes.

Key Classes and Responsibilities

- **User:** Stores user credentials, profile information, and roles.
- **Project:** Represents a collaborative coding project.
- **Session:** Manages an active collaboration instance.
- **Participant:** Represents a user's involvement in a session.
- **CodeDocument:** Stores shared code content and metadata.
- **Message:** Represents chat messages and signaling data.

Relationships and Multiplicity

Users can own or participate in multiple projects. Projects may host multiple sessions, and each session contains multiple participants. These relationships enforce logical collaboration boundaries and ensure data consistency.



3.2.2 Persistent Model

The **persistent model** of CodeCollab defines how system objects are stored and managed in long-term storage using an **object-oriented persistence approach**. Unlike traditional relational database design using E-R diagrams, this approach maps classes and their relationships directly to persistent storage, allowing objects in the system to be saved and retrieved with minimal impedance mismatch. The persistent model is essential for ensuring **data consistency, integrity, and efficient access** throughout the lifecycle of projects and collaboration sessions.

Each **persistent class** in the system includes the following elements:

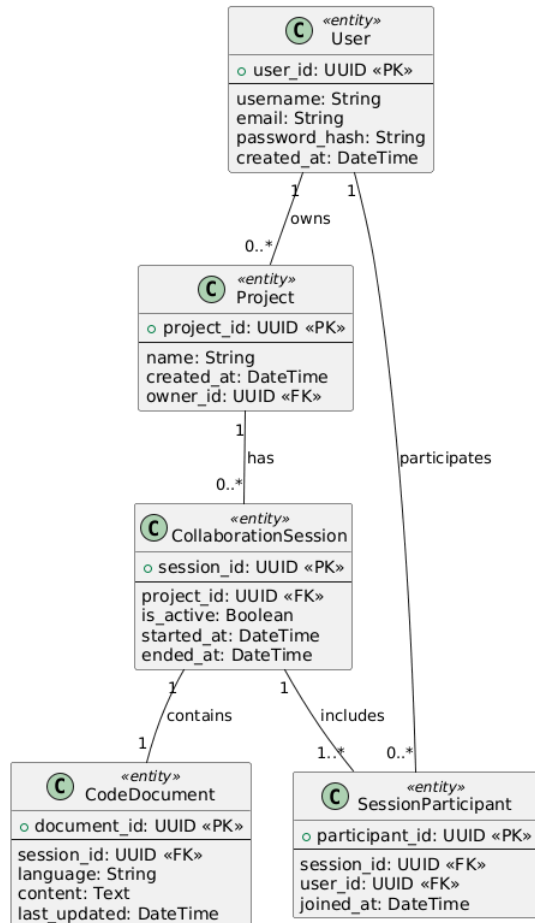
- **Attributes with data types and lengths:** Each class attribute is defined with a specific data type (e.g., string, integer, boolean, timestamp) and, where applicable, a maximum length. For example, the User class may have username: String(50) and email: String(100) to control storage and enforce data constraints. This ensures predictable storage requirements and prevents invalid data from being persisted.
- **Primary keys for unique identification:** Every persistent class has a unique identifier that serves as a primary key. For instance, the Project class may have a project_id field that uniquely identifies each project. Primary keys are crucial for distinguishing objects in the database and for maintaining **referential integrity**.

- **Foreign keys to represent relationships:** Relationships between classes are represented using foreign keys. For example, each Session object may contain a project_id foreign key linking it to its parent Project, and each Participant may reference both a user_id and a session_id. This allows the system to maintain **consistent relationships** between objects while enforcing constraints on dependent data.
- **Multiplicity and role definitions:** The persistent model defines how many instances of one class can be associated with another. For example, a single Project can have multiple Session objects (1-to-many), while each Session may have multiple Participant objects. Role definitions clarify the function of each association, such as “owner,” “member,” or “observer.”

The persistent model ensures a **seamless mapping between in-memory objects and stored data**, allowing the backend to efficiently save and retrieve objects without losing their structure or relationships. This approach also enables **efficient query operations**, simplifies **transaction management**, and supports **scalable and maintainable system architecture**.

By carefully defining classes, attributes, keys, and relationships, the persistent model ensures that the system maintains **data integrity**, supports collaborative features like live editing, and allows recovery from unexpected failures while keeping all user and session data consistent.

CodeCollab - Persistent Model

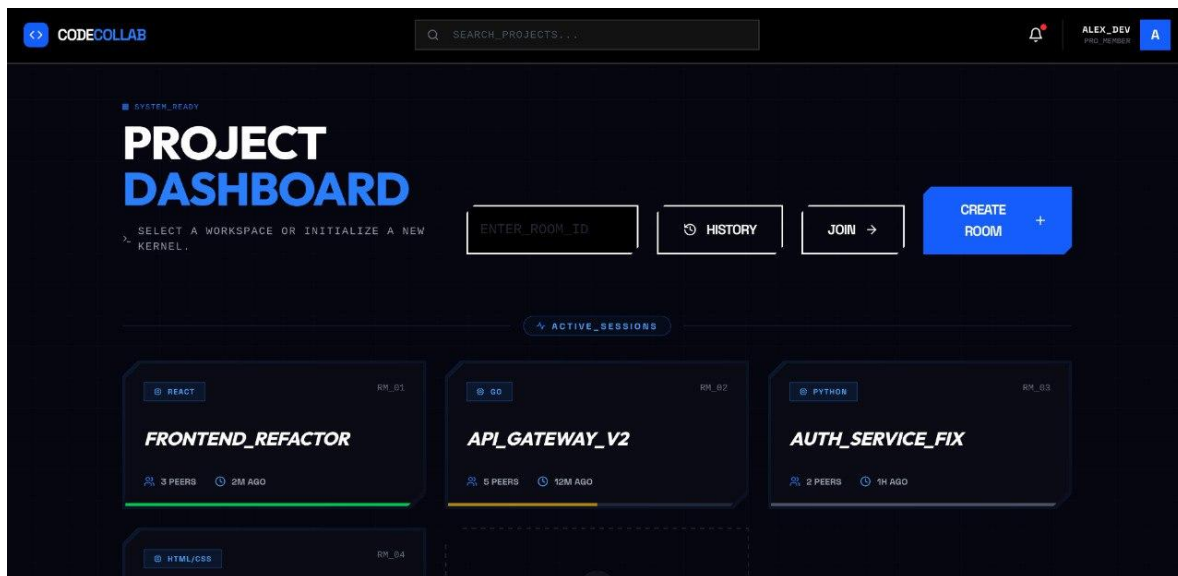


3.3 User Interface Design

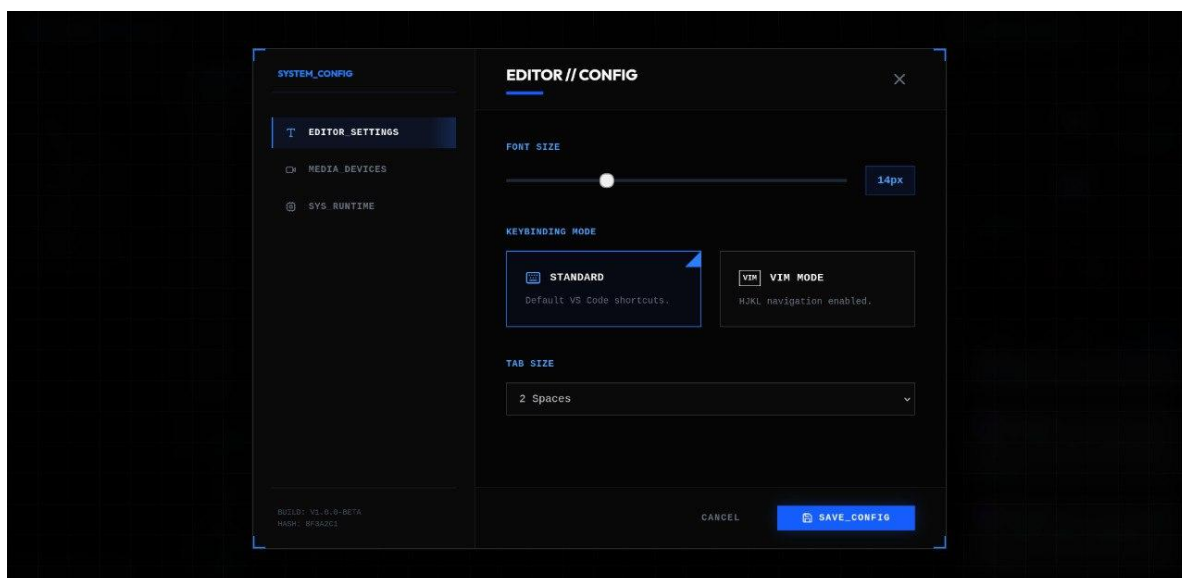
The user interface (UI) of CodeCollab is designed to provide a **clean, responsive, and intuitive experience** for users engaging in real-time collaborative software development. The UI is implemented using **React.js**, which allows a **component-based architecture**, ensuring modularity and reusability of interface elements. This approach enables each module to be developed, tested, and updated independently while maintaining a consistent look and feel throughout the platform.

Currently, the CodeCollab UI comprises **seven main modules**:

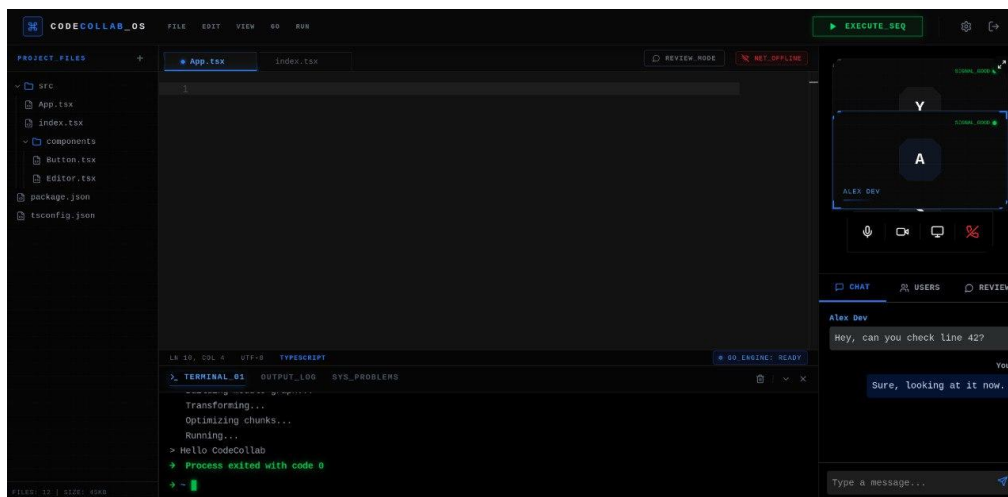
Dashboard: The central landing page that provides an overview of projects, active collaboration sessions, notifications, and recent activities. Users can quickly navigate to projects or sessions, allowing for efficient management of their work environment.



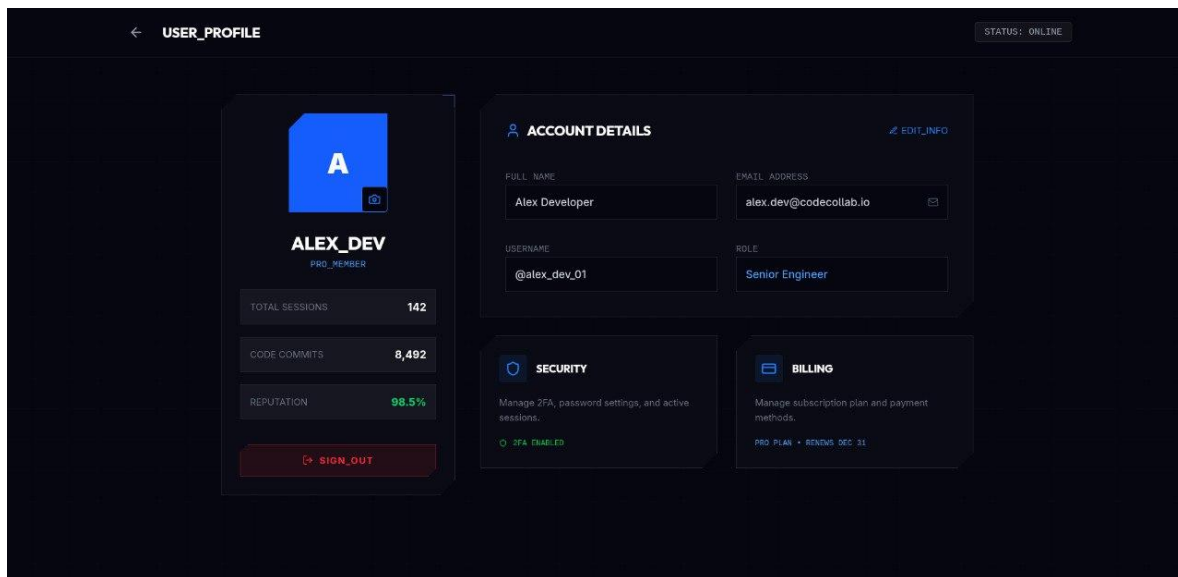
Settings: Allows users to configure personal preferences, account details, notification options, and editor settings. Users can customize themes, manage privacy settings, and configure editor features to create a tailored and comfortable development experience.



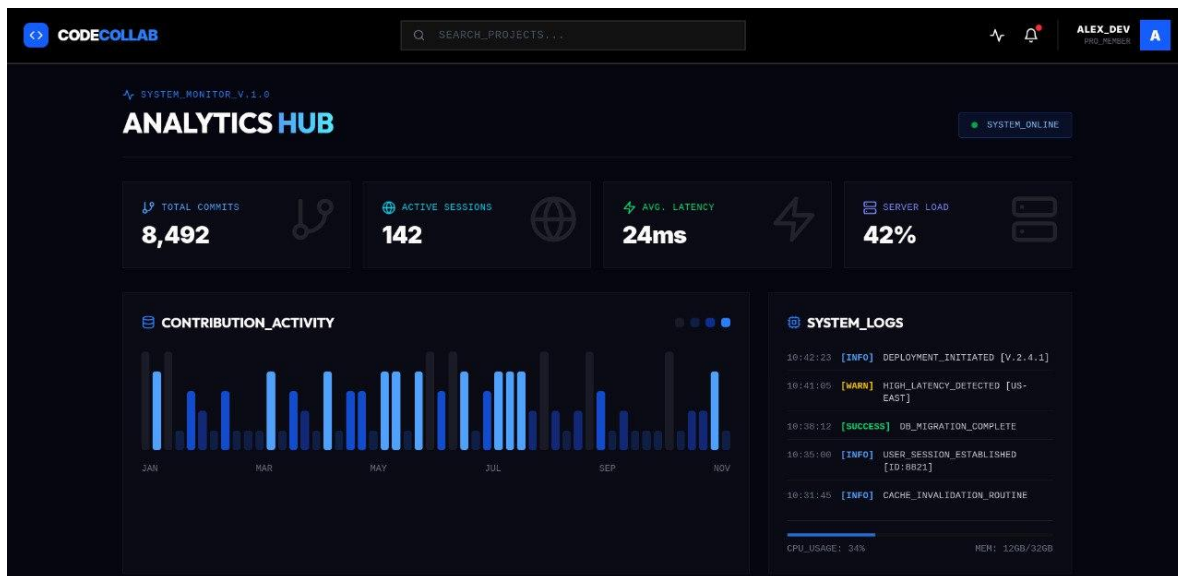
Editor: The core workspace integrating the **Monaco Editor**, providing advanced code editing capabilities such as syntax highlighting, IntelliSense, and real-time collaborative editing. Multiple users can work on the same code document simultaneously, with changes synchronized immediately across all participants.



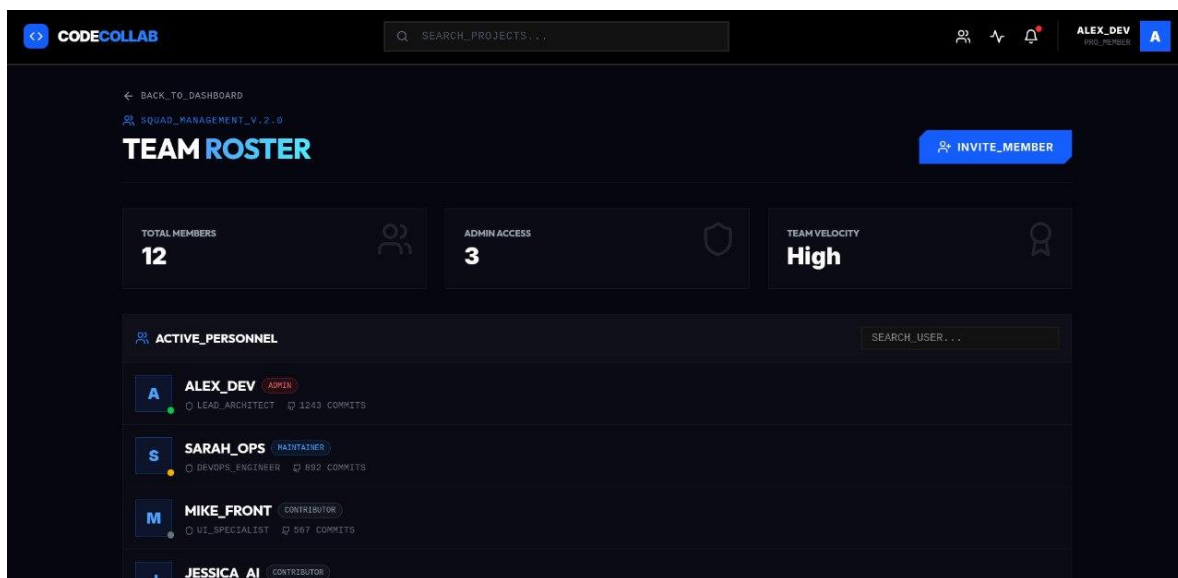
User Account: Manages user profiles, authentication credentials, and role information. Users can update their personal information, view their activity history, and manage access permissions associated with their account.



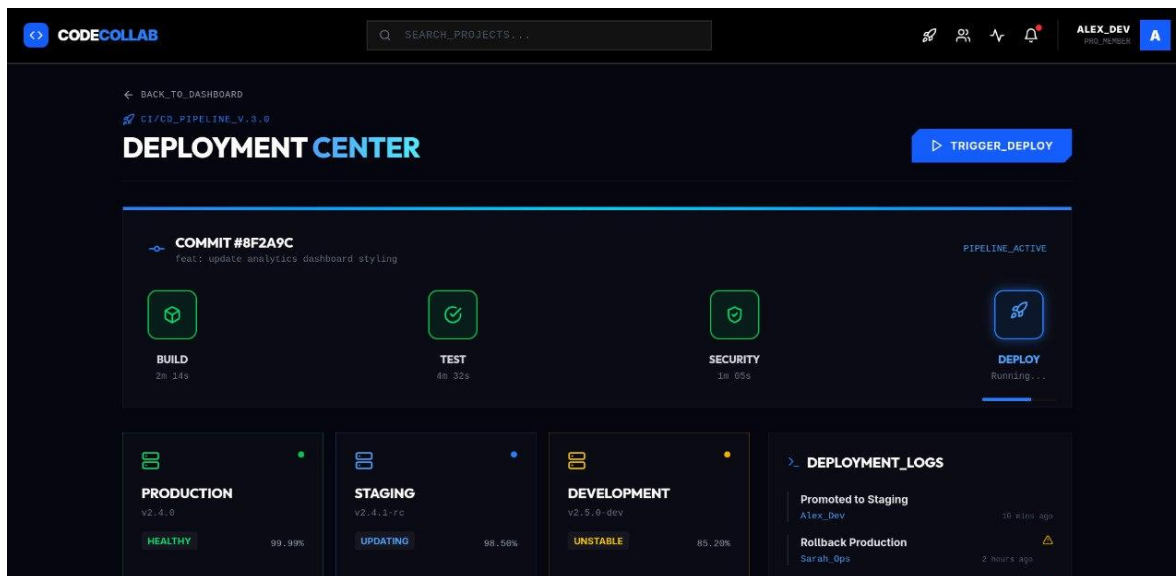
Analytics: Displays project-related insights and collaboration metrics. This module provides statistics such as code contributions, session durations, user activity levels, and project performance trends, supporting team leads and administrators in monitoring productivity.



Team Management: Enables administrators and project owners to manage project teams. Users can be added or removed from projects, assigned specific roles, and granted appropriate permissions. This module ensures consistent enforcement of access control and collaboration policies.



Deployment UI: Provides a visual interface for managing system deployment and cloud resources. Users (primarily administrators or DevOps personnel) can monitor server status, deploy updates, view container status, and configure environment variables. This module integrates deployment management with the platform interface, improving operational efficiency and reducing the need for external deployment tools.



The design of these modules emphasizes **usability, clarity, and seamless interaction**. Navigation is intuitive, with consistent menus and clear labeling to minimize cognitive load. Real-time visual feedback ensures that user actions, such as code edits or session updates, are immediately reflected across the interface. The responsive layout adapts to various screen sizes, including desktops and tablets, providing a consistent collaborative experience. By integrating development, management, analytics, and deployment tools into a **single cohesive interface**, CodeCollab ensures that users and administrators can perform all necessary tasks within one unified environment, simulating the collaborative dynamics of a physical workspace.

3.4 Access Control and Security

Security and access control are critical components of the CodeCollab system, as the platform manages sensitive user information, collaborative source code, and real-time communication streams. Ensuring that only authorized users can access system resources is essential to maintain the **confidentiality, integrity, and availability** of the system. CodeCollab employs a **layered security approach**, combining **role-based access control, secure authentication mechanisms, encrypted communications, and session and data protection** to safeguard the platform.

3.4.1 User Roles and Privileges

The system adopts a **Role-Based Access Control (RBAC)** model, where access to features and data depends on the user's assigned role. CodeCollab defines three primary roles:

- **Administrator:** The administrator has the highest level of access and is responsible for system management. This role includes creating and managing user accounts, assigning

roles, configuring system settings, monitoring system usage, and reviewing activity logs. Administrators have the authority to manage all projects, sessions, and user permissions across the platform.

- **Authenticated User:** Authenticated users are registered and verified individuals who actively participate in the system. Their privileges include creating and editing projects, initiating and joining real-time collaboration sessions, contributing code, participating in audio and video communication, and managing their account settings. Access to projects and sessions is restricted to those for which they are authorized.
- **Guest User:** Guest users have limited access, primarily for observation or demonstration purposes. Guests can view shared projects or sessions in a read-only mode and may participate in audio/video communication if granted permission by session owners. They cannot modify code, create projects, or access persistent data, ensuring sensitive resources are protected.

By clearly defining roles and responsibilities, CodeCollab prevents unauthorized access and minimizes the risk of accidental or malicious misuse of system functionality.

3.4.2 Authentication Mechanism

CodeCollab implements **JSON Web Tokens (JWT)** for secure, stateless authentication. The process includes:

- ✓ Users provide credentials during login.
- ✓ Upon successful verification, the backend generates a **signed JWT**, which encodes the user's identity, role, and session information.
- ✓ The token is stored securely on the client side and sent with every request to validate the user's identity.
- ✓ The backend verifies the token's integrity before granting access to requested resources.

This approach eliminates the need for server-side session storage, supports horizontal scaling, and ensures that only authenticated users can perform system operations.

3.4.3 Authorization and Access Control

Authorization is enforced through **RBAC**, ensuring that users can only access resources and perform actions allowed by their role:

- ❖ API endpoints are protected based on user roles.
- ❖ Session-level checks prevent unauthorized users from joining active collaboration sessions.
- ❖ Backend services enforce permissions for creating, editing, or deleting projects and sessions.
- ❖ Team management functions are restricted to project owners or administrators.

By combining authentication and role-based authorization, the system enforces **fine-grained access control** and prevents privilege escalation.

3.4.4 Secure Communication

All interactions between clients and backend services, as well as real-time collaboration communication, are encrypted:

- ✓ **HTTPS** secures REST API requests.
- ✓ **WSS (WebSocket Secure)** protects real-time code synchronization.
- ✓ **WebRTC secure channels** (DTLS/SRTP) encrypt peer-to-peer audio and video streams.

Encrypted communication ensures that sensitive information, such as project code or session metadata, cannot be intercepted or tampered with during transmission.

3.4.5 Session and Data Security

Session security ensures that collaboration sessions remain private and properly isolated:

- ✓ Each session is uniquely identified with a secure ID.
- ✓ Only authenticated and authorized participants can join.
- ✓ Session timeouts and reconnection mechanisms prevent unauthorized reuse of session resources.
- ✓ Persistent data in the database is accessible only through the backend, preventing direct unauthorized access.
- ✓ Passwords and other sensitive information are stored using secure hashing techniques.

These measures protect against data leakage, unauthorized code changes, and accidental or malicious manipulation of collaboration sessions.